UNIT -1

Artificial Neural Network

Soft Computing(TCS821)

by

Dr. Manoj Singh Adhikari

Subject Code: TCS 821

Course Outcomes: After completion of the course students will be able to

1. Summarize about soft computing techniques and their applications
2. Analyze various neural network architectures
3. Design perceptrons and counter propagation networks.
4. Classify the fuzzy systems
5. Analyze the genetic algorithms and their applications.
6. Compose the fuzzy rules.

UNIT	CONTENTS
Unit - I	Fundamentals of ANN: The Biological Neural Network, Artificial Neural Networks -Building Blocks of ANN and ANN terminologies: architecture, setting of weights,activation functions - McCulloch-pitts Neuron Model, Hebbian Learning rule, Perceptionlearning rule, Delta learning rule.
Unit - II	Models of ANN: Single layer perception, Architecture, Algorithm, application procedure- Feedback Networks: Hopfield Net and BAM - Feed Forward Networks: BackPropogation Network (BPN) and Radial Basis Function Network (RBFN) – SelfOrganizing Feature Maps: SOM and LVQ
Unit – III	Fuzzy Sets, properties and operations - Fuzzy relations, cardinality, operations andproperties of fuzzy relations, fuzzy composition.
Unit – IV	Fuzzy variables - Types of membership functions - fuzzy rules: Takagi and Mamdani –fuzzy inference systems: fuzzification, inference, rulebase, defuzzification.
Unit – V	Genetic Algorithm (GA): Biological terminology – elements of GA: encoding, types ofselection, types of crossover, mutation, reinsertion – a simple genetic algorithm –Theoretical foundation: schema, fundamental theorem of GA, building block hypothesis.

ANN

Explain human brain with basic component. (CO1)

The **human brain** is the main control center of the body. It controls thinking, memory, emotions, movement, breathing, heartbeat, and other vital functions. It is protected by the skull and is a key part of the **central nervous system**.

Basic Components of the Human Brain

1. Cerebrum

Largest part of the brain.
Divided into **left and right hemispheres**.
Responsible for:

- Thinking and reasoning
- Memory and learning
- Intelligence
- Voluntary actions (walking, writing, speaking)

Contains four lobes: **frontal, parietal, temporal, occipital**.

2. Cerebellum

Located at the back of the brain, below the cerebrum.

Controls:

- Balance and posture
- Coordination of muscles
- Precision and smooth movement

Important for motor learning.

3. Brainstem

Connects the brain to the spinal cord.

Controls **involuntary activities** such as:

- Breathing
- Heartbeat
- Blood pressure
- Swallowing

Made up of:

- **Midbrain**
- **Pons**
- **Medulla oblongata**

4. Thalamus

Acts as a relay station.
Transfers sensory information (except smell) to the cerebrum.
Helps in alertness and consciousness.

5. Hypothalamus

Maintains body balance (**homeostasis**).
Controls:

- Body temperature
- Hunger and thirst
- Sleep cycle
- Hormone secretion (via pituitary gland)

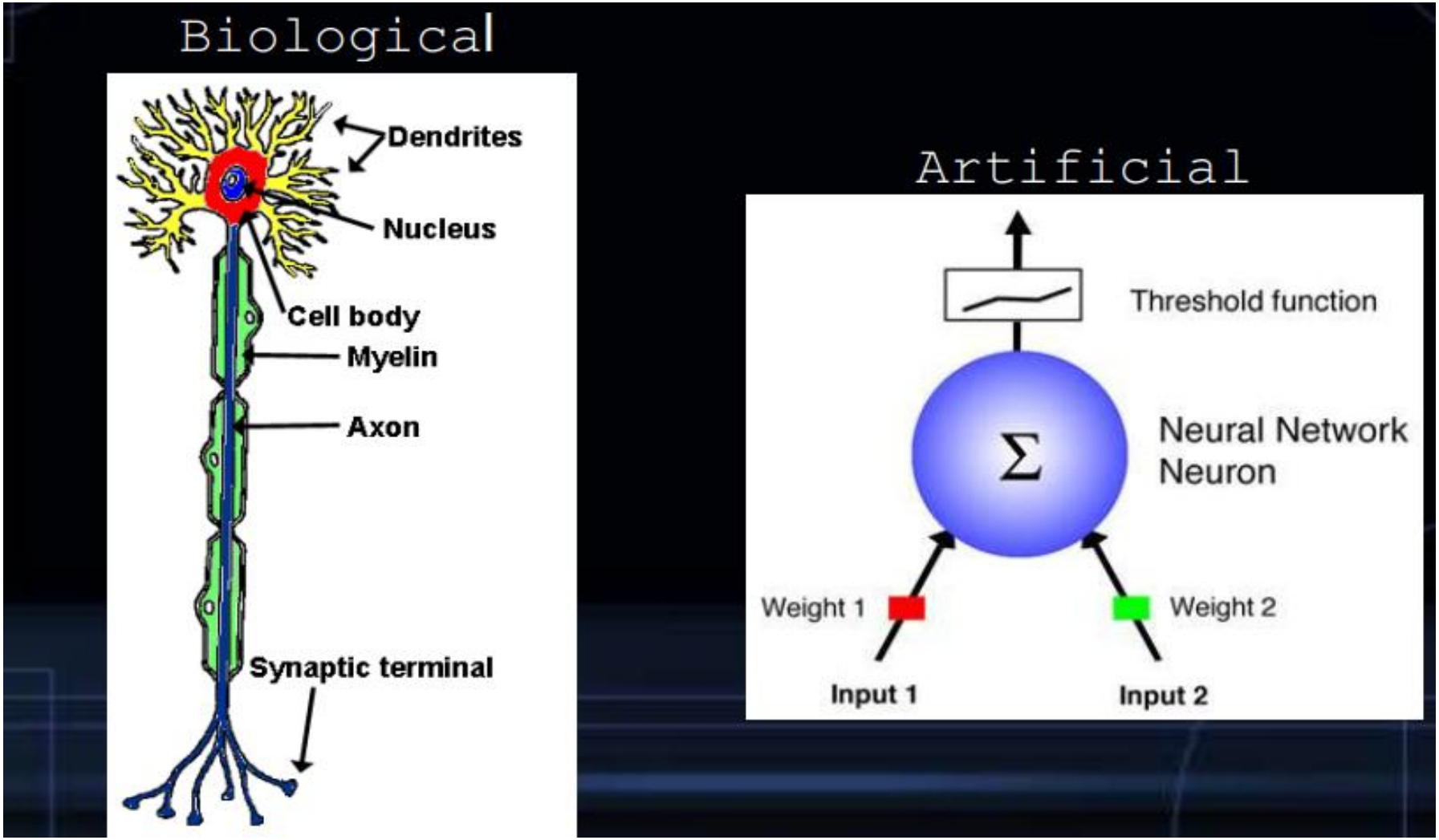
Basic Components of the Human Brain

Component	Main Function
Cerebrum	Thinking, memory, voluntary actions
Cerebellum	Balance, coordination
Brainstem	Breathing, heartbeat
Thalamus	Sensory signal relay
Hypothalamus	Body regulation & hormones

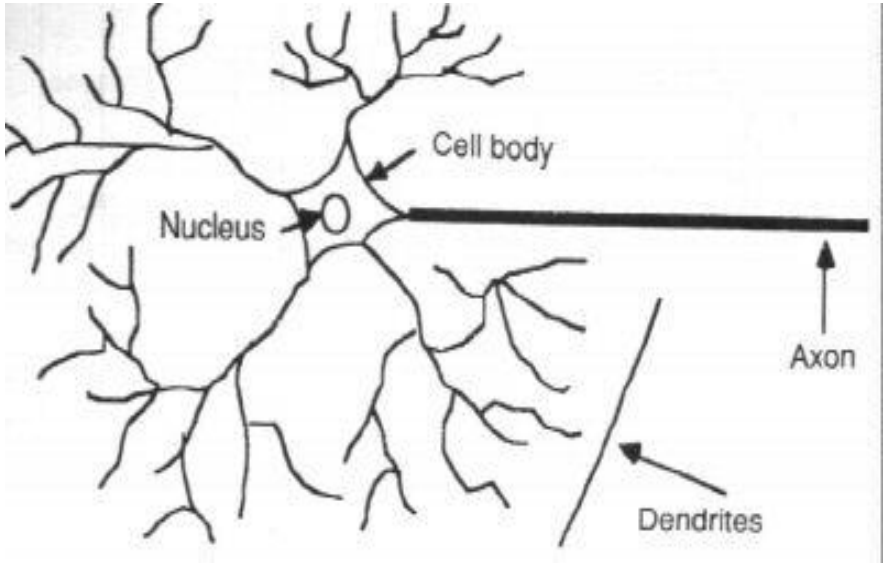
Differentiate between Hard and Soft Computing. (CO1)

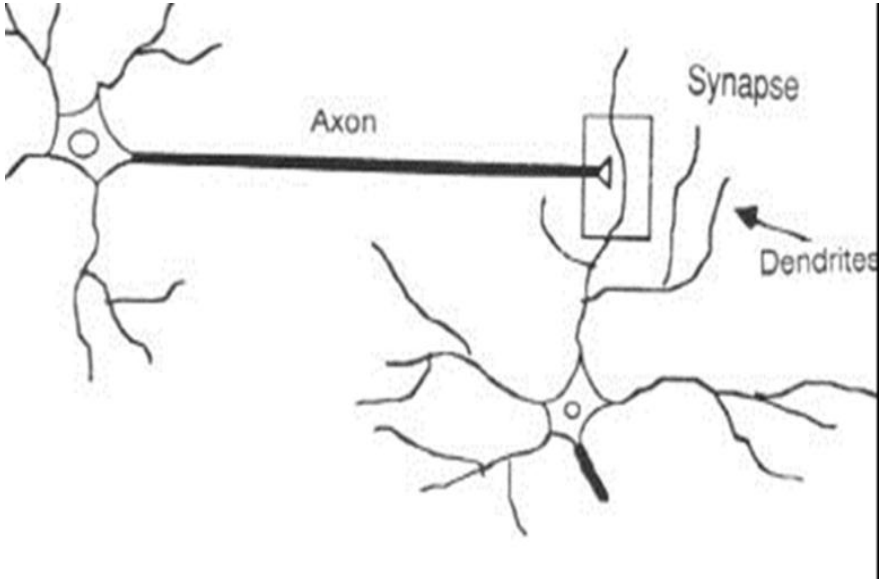
Aspect	Hard Computing	Soft Computing
Definition	Hard computing is based on precise, exact, and deterministic algorithms.	Soft computing is based on approximation, uncertainty, and tolerance to imprecision.
Nature of Solution	Produces exact and accurate results.	Produces approximate but acceptable results.
Logic Used	Uses classical logic (Boolean logic: true/false).	Uses fuzzy logic, probabilistic reasoning, and learning methods.
Input Requirements	Requires complete, precise, and accurate input data.	Can work with incomplete, noisy, or uncertain data.
Flexibility	Less flexible and rigid in nature.	Highly flexible and adaptive.
Learning Ability	No learning capability.	Has learning and self-adaptation capability.
Examples	Conventional algorithms, numerical methods, digital computers.	Fuzzy Logic, Neural Networks, Genetic Algorithms, Machine Learning.
Applications	Banking calculations, compiler design, scientific computations.	Pattern recognition, speech recognition, robotics, control systems.

Biological approach to AI



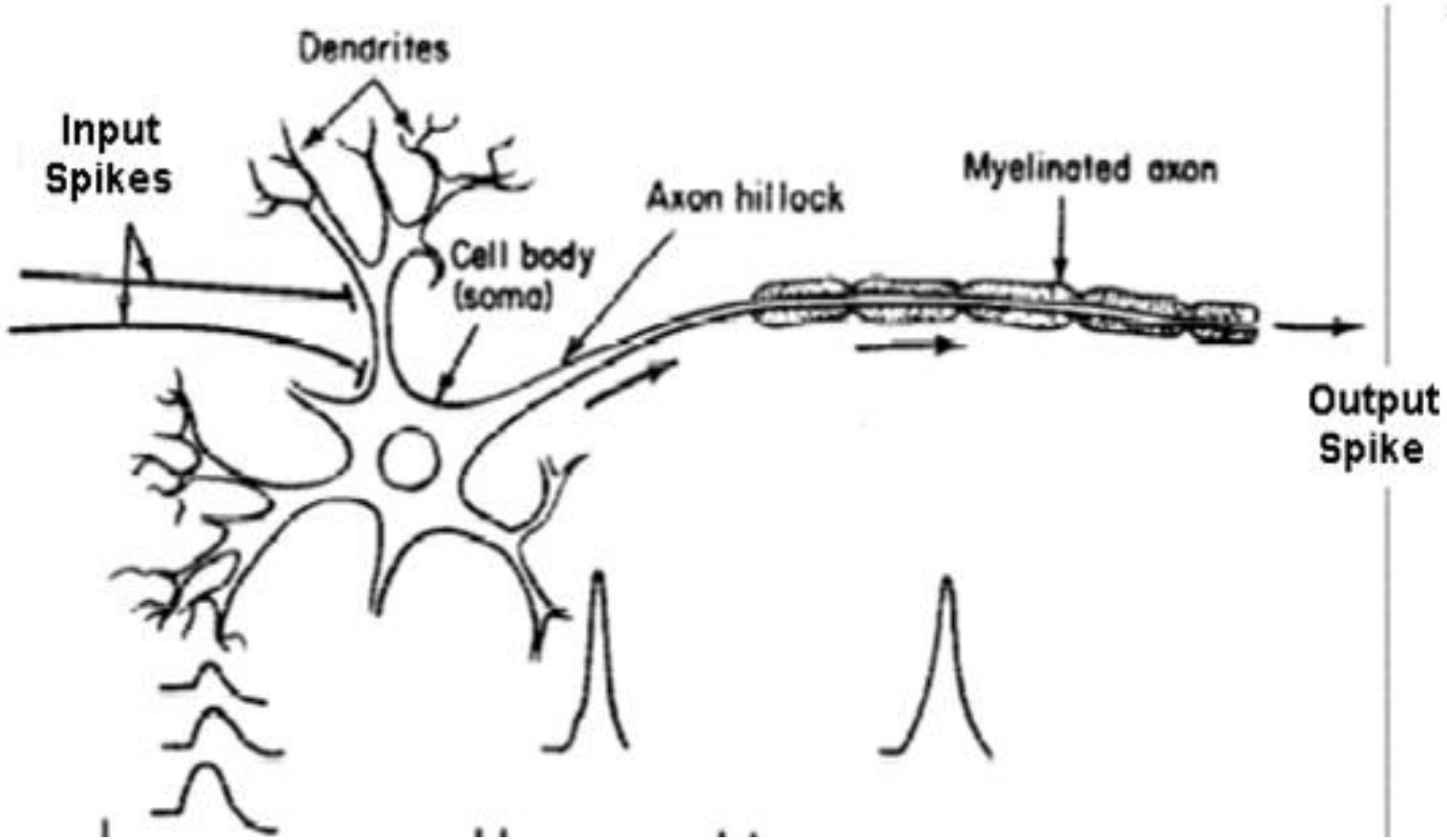
- Natural neurons





The information transmission happens at the synapses.

Basic Input-Output Transformation



BASICS OF NEURAL NETWORK

- Biological approach to AI
- Developed in 1943
- Comprised of one or more layers of neurons
- Several types, we will focus on feed-forward and feedback networks

From Biological to Artificial Neurons

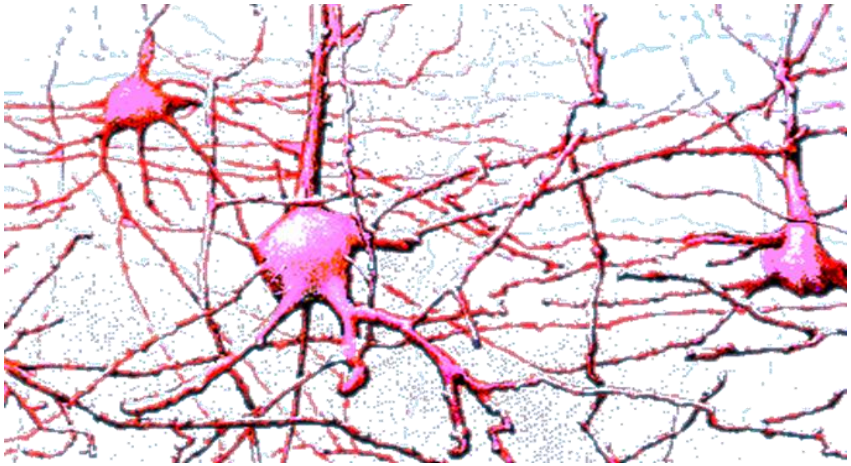
- The Neuron - A Biological Information Processor
- dendrites - the receivers
- soma - neuron cell body (sums input signals)
- axon - the transmitter
- synapse - point of transmission
- neuron activates after a certain threshold is meet
- Learning occurs via electro-chemical changes in effectiveness of synaptic junction.

Introduction

- Artificial Neural Networks (ANN)
  - Information processing inspired by biological nervous systems
  - ANN is composed of a system of neurons connected by synapses
  - ANN learn by
    - Adjust synaptic connections between neurons

Biological Neuron

- Animals are able to react adaptively to changes in their external and internal environment, and they use their nervous system to perform these behaviours.
- An appropriate model/simulation of the nervous system should be able to produce similar responses and behaviours in artificial systems.



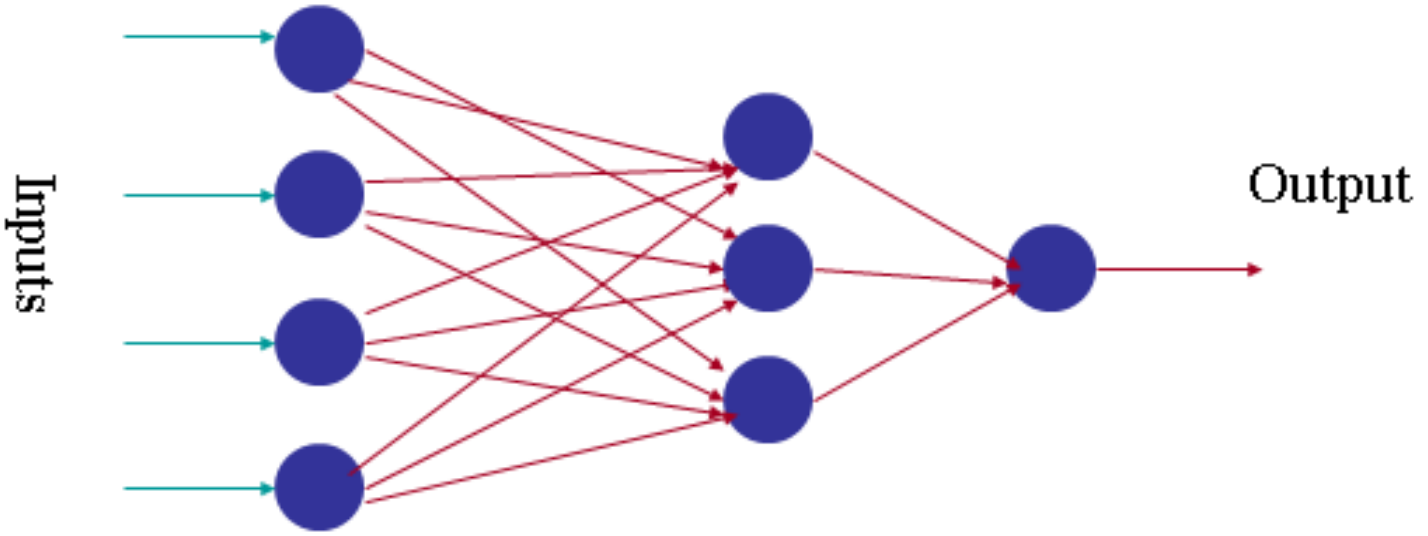
Artificial neural networks

- An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the biological nervous systems, such as the human brain's information processing mechanism.

- The key element of this network is the novel structure of the information processing system. It is composed of a large number of highly interconnected processing elements (neurons) working in unison to solve specific problems.

- An NN is configured for a specific application, such as pattern recognition or data classification, through a learning process. Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons.

Artificial neural networks



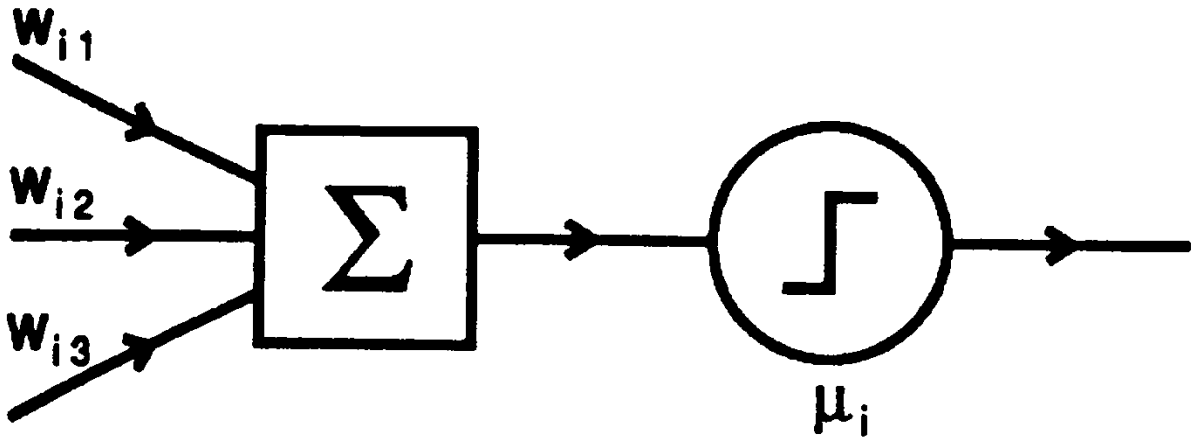
An artificial neural network is composed of many artificial neurons that are linked together according to a specific network architecture. The objective of the neural network is to transform the inputs into meaningful outputs.

Artificial neural networks

Tasks to be solved by Artificial Neural Networks:-

- Controlling the movements of a robot based on self-perception and other information (e.g., visual information)
- Deciding the category of potential food items (e.g., edible or non-edible) in an artificial world
- Recognizing a visual object (e.g., a familiar face)
- Predicting where a moving object goes, when a robot wants to catch it.

- Attributes of neuron
  - m binary inputs and 1 output (0 or 1)
  - Synaptic weights w_{ij}
  - Threshold μ_i



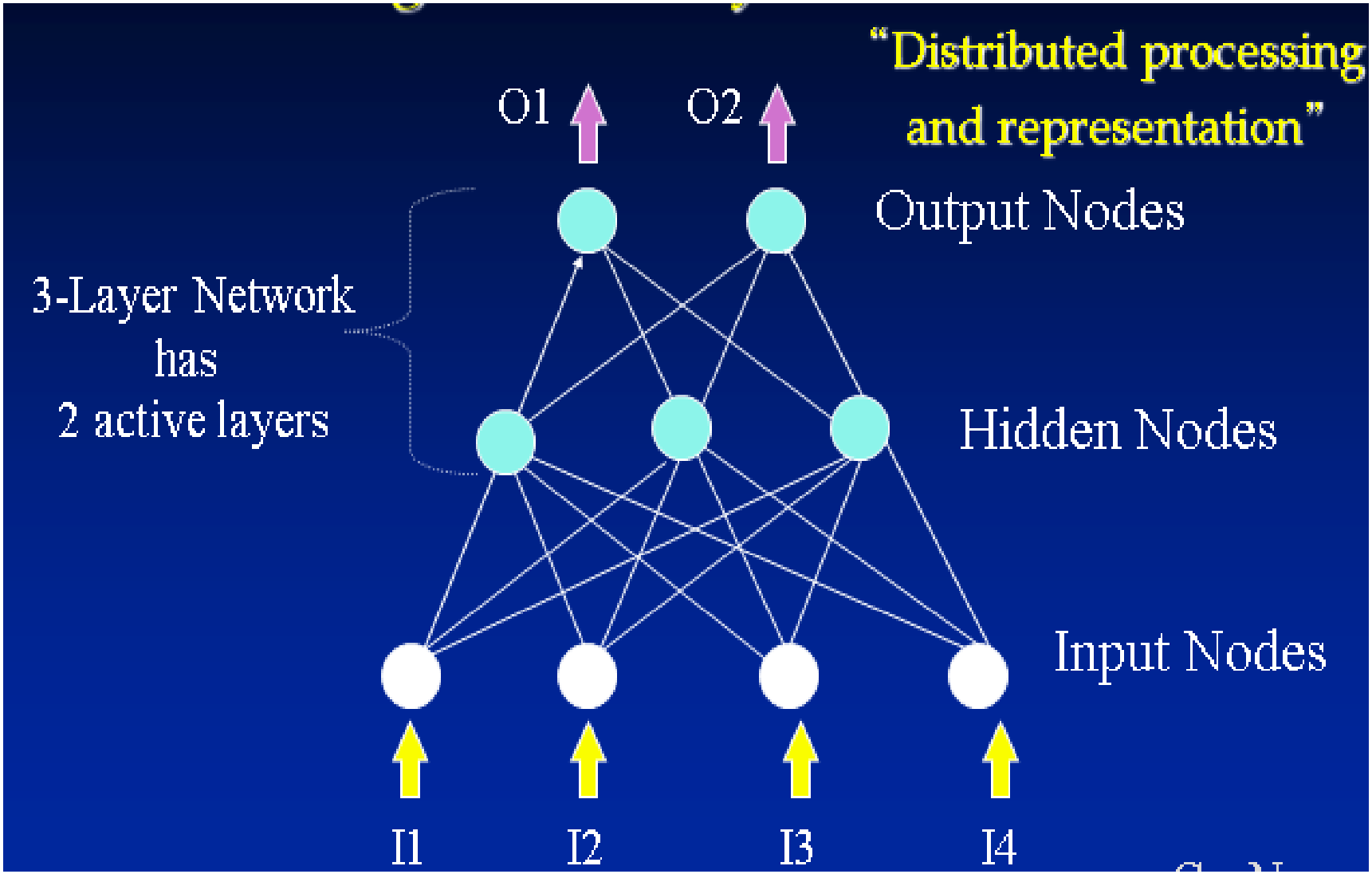
- An Artificial Neuron - The Perceptron

  simulated on hardware/software
- input connections - the receivers
- node, and unit simulates neuron body
- output connection - the transmitter
- activation function employs a threshold or bias
- connection weights act as synaptic junctions
- Learning occurs via changes in value of the connection weights.

- Basic function of neuron is to sum inputs, and produce output, given sum is greater than threshold

- ANN node produces an output as follows:

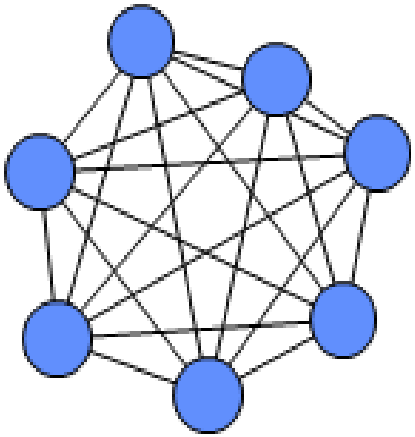
1. Multiplies each component of the input pattern by the weight of its connection
2. Sums all weighted inputs
3. Transforms the total weighted input into the output using the activation function


"Distributed processing and representation"
O1
O2
Output Nodes
3-Layer Network has 2 active layers
Hidden Nodes
Input Nodes
I1
I2
I3
I4

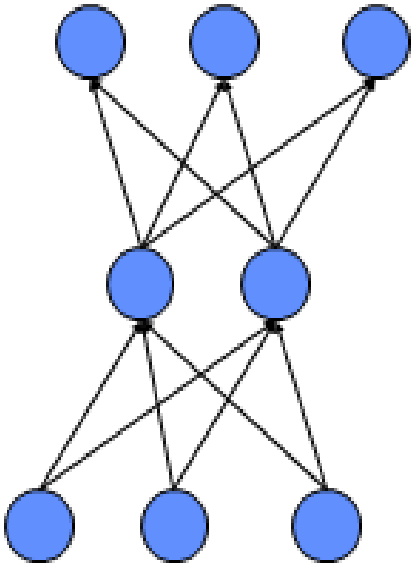
Behaviour of an artificial neural network to any particular input depends upon:-

- structure of each node (activation function)
- structure of the network (architecture)
- weights on each of the connections

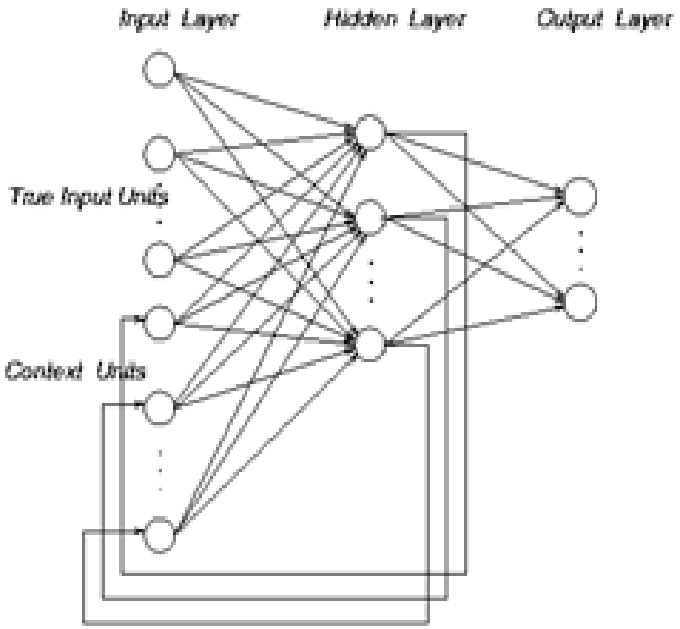
Topologies of Neural Networks



completely connected



feedforward (directed, a-cyclic)



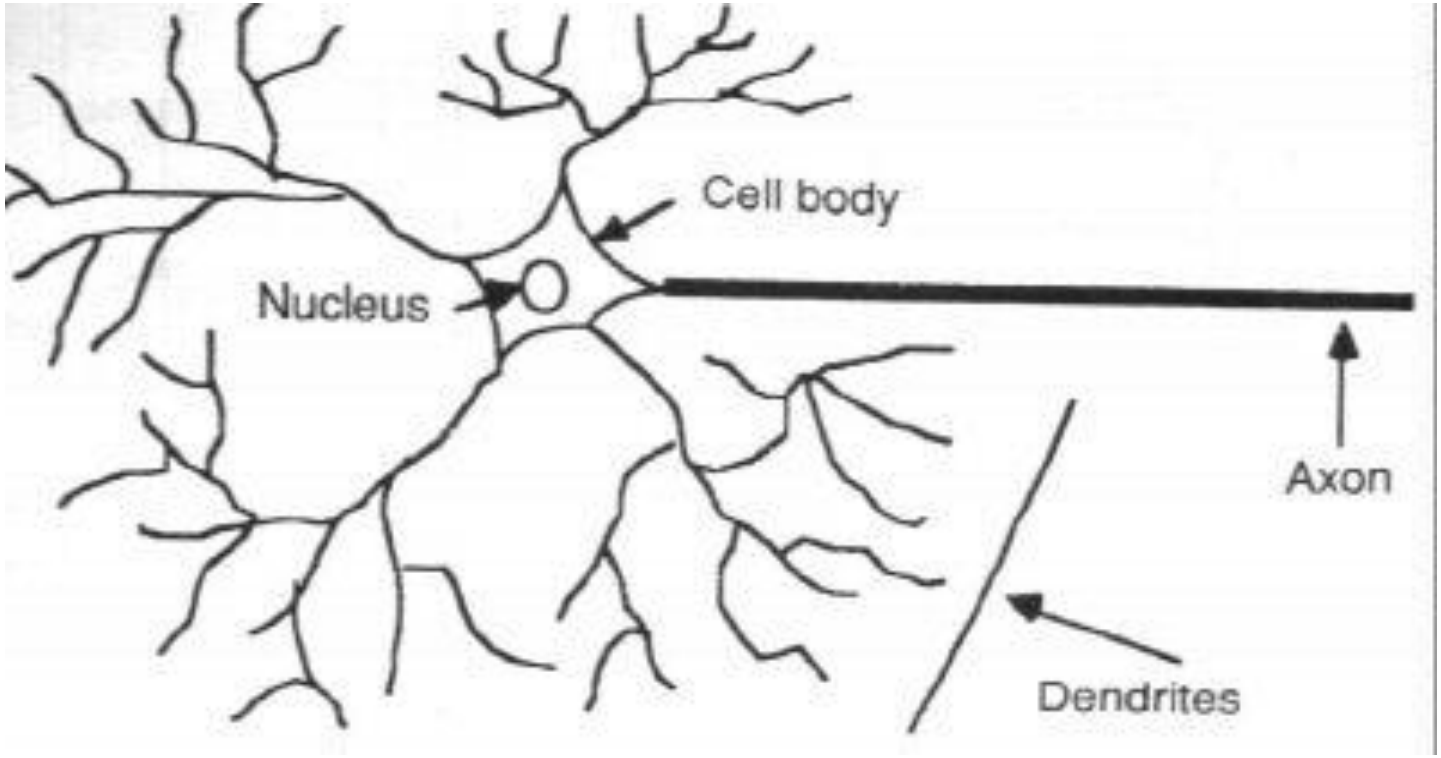
recurrent (feedback connections)

Neuron Model

- Neuron collects signals from *dendrites*
- Sends out spikes of electrical activity through an *axon*, which splits into thousands of branches.
- At end of each brand, a *synapses* converts activity into either exciting or inhibiting activity of a dendrite at another neuron.
- Neuron *fires* when exciting activity surpasses inhibitory activity
- Learning changes the effectiveness of the synapses

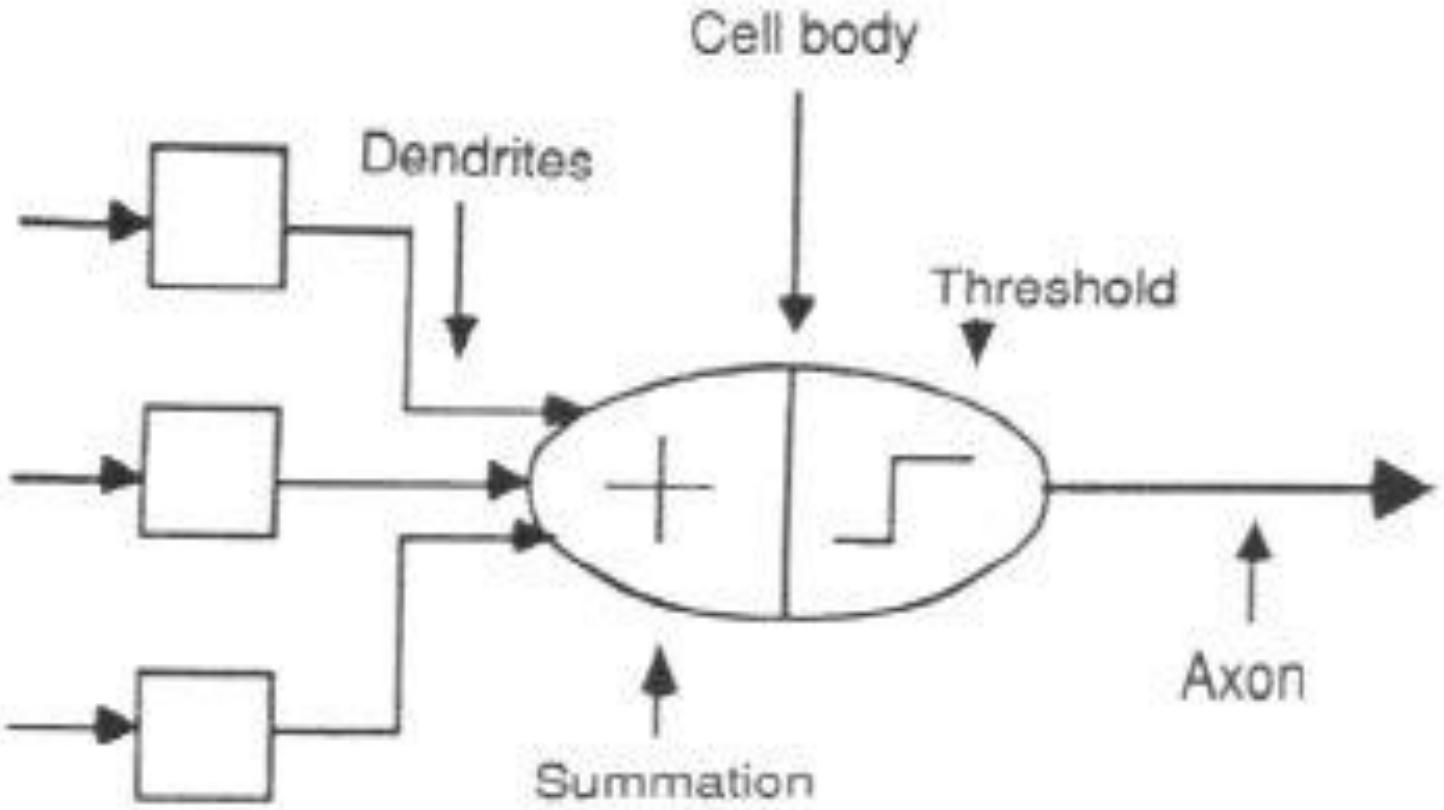
Neuron Model

- Natural neurons

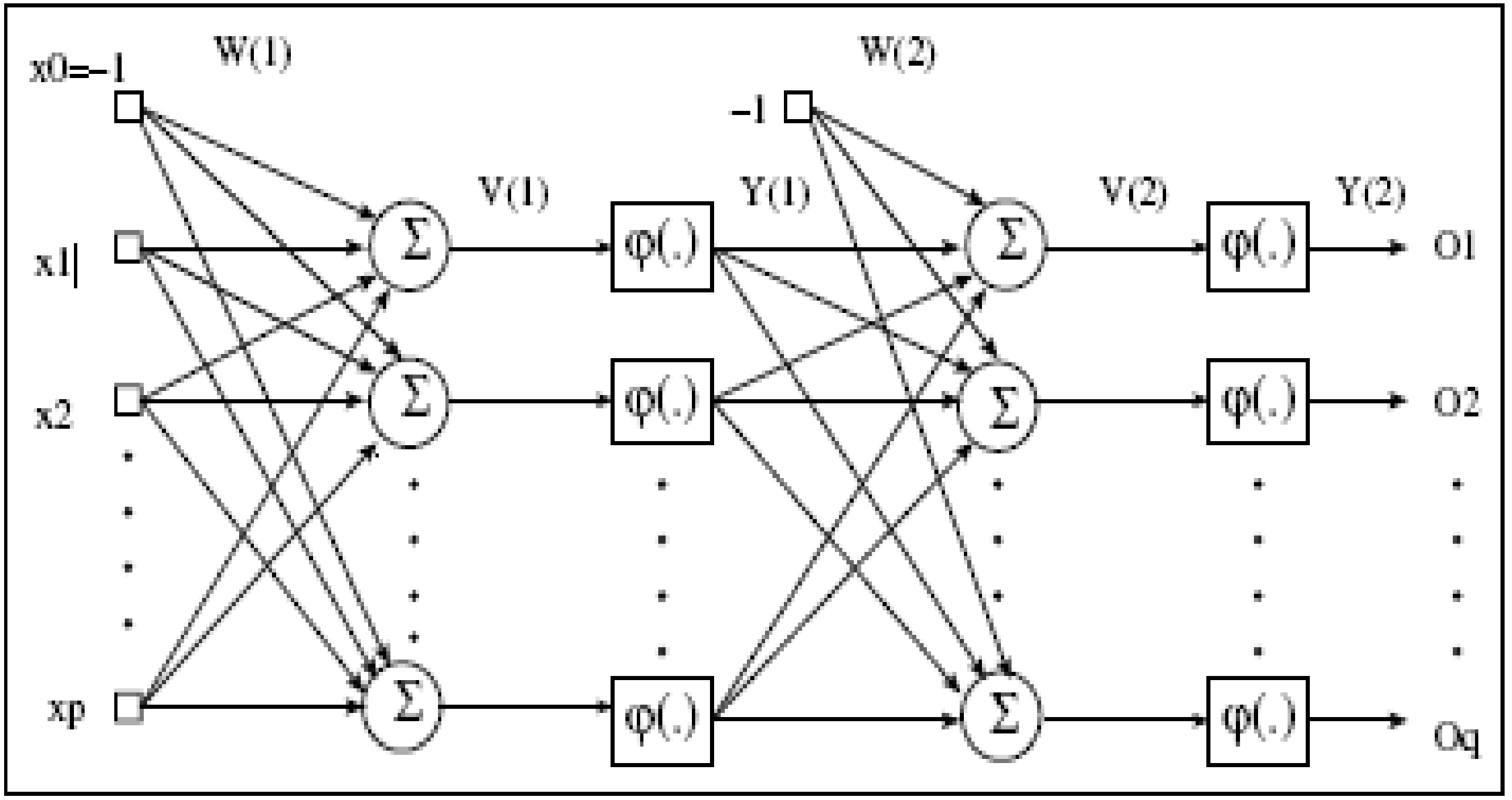


Neuron Model

- Abstract neuron model:



ANN Forward Propagation



ANN Forward Propagation

- Bias Nodes
  - Add one node to each layer that has constant output
- Forward propagation
  - Calculate from input layer to output layer
  - For each neuron:
    - Calculate weighted average of input
    - Calculate activation function

History

- 1943: McCulloch–Pitts "neuron"
  Started the field
- 1962: Rosenblatt's perceptron
  Learned its own weight values; convergence proof
- 1969: Minsky & Papert book on perceptrons
  Proved limitations of single-layer perceptron networks
- 1982: Hopfield and convergence in symmetric networks
  Introduced energy-function concept
- 1986: Backpropagation of errors
  Method for training multilayer networks
- Present: Probabilistic interpretations, and spiking networks

History

- **1943:** McCulloch and Pitts model neural networks **based on their understanding of neurology.**
  - Neurons consists simple logic functions:
    - a or b
    - a and b

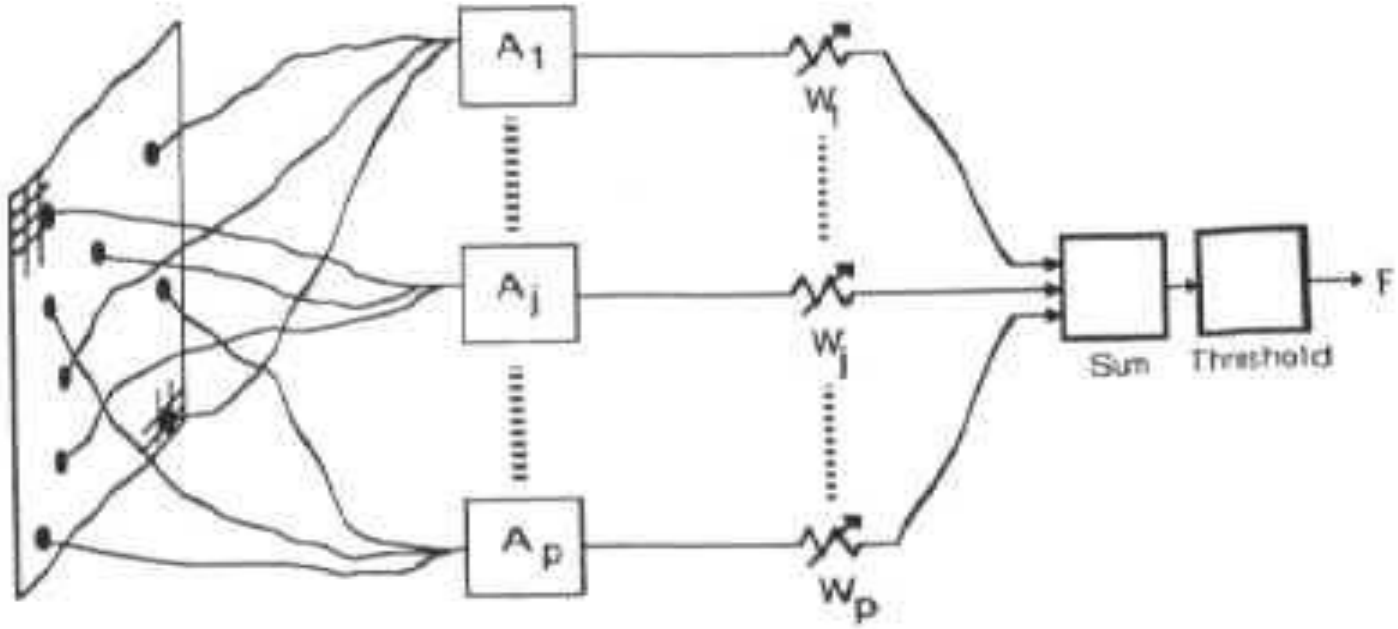
History

- Perceptron (Rosenblatt 1958)
  - Three layer system:
    - Input nodes
    - Output node
    - Association layer
  - Can learn to connect or associate a given input to a random output unit

- Minsky and Papert
  - Showed that a single layer perceptron cannot learn the XOR of two binary inputs

History

- Perceptron (Rosenblatt 1958)
  - Association units A_1, A_2, … extract features from user input
  - Output is weighted and associated
  - Function fires if weighted sum of input exceeds a threshold.



History

- **Back-propagation learning method (Werbos 1974)**
  - Three layers of neurons
    - Input, Output, Hidden
  - Better learning rule for generic three layer networks
  - Regenerates interest in the 1980s
- Successful applications in medicine, marketing, risk management.

Explain the types of neural network.(CO1)

Neural networks can be classified into several types based on their architecture and learning process. The main types are explained below:

1. Feedforward Neural Network (FNN)
Information flows only in one direction: from input layer → hidden layer(s) → output layer.
No feedback or loops.
Used in pattern recognition and classification.
Example: **Perceptron**, **Multilayer Perceptron (MLP)**.

2. Convolutional Neural Network (CNN)
Specially designed for image and video processing.
Uses convolution, pooling, and fully connected layers.
Automatically extracts features from images.
Applications: image recognition, medical imaging, object detection.

Explain the types of neural network.(CO1)

3. Recurrent Neural Network (RNN)
Has feedback connections (loops).
Can store previous information (memory).
Suitable for sequential data.
Applications: speech recognition, time-series prediction, NLP.
Variants: **LSTM (Long Short-Term Memory)**, **GRU (Gated Recurrent Unit)**.

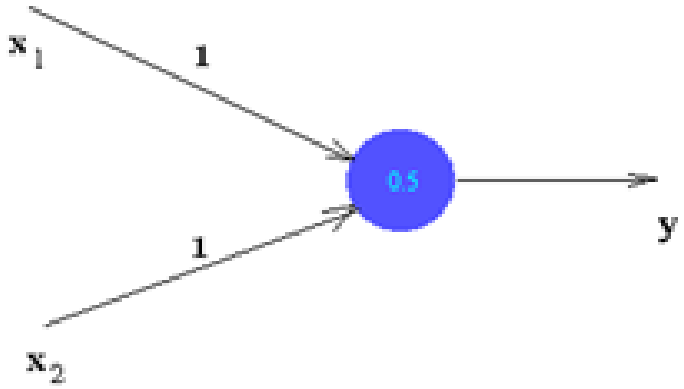
4. Radial Basis Function Network (RBFN)
Uses radial basis functions as activation functions.
Typically has three layers: input, hidden, and output.
Fast training and good for function approximation.

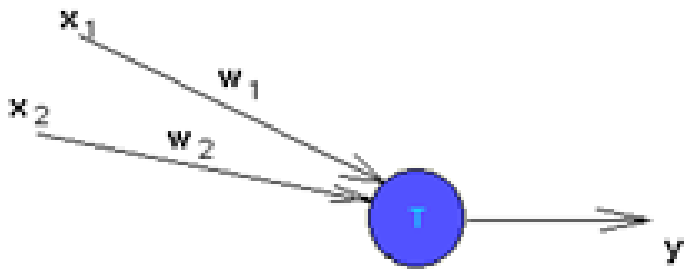
Explain the types of neural network.(CO1)

5. Artificial Neural Network (ANN)
General term for neural networks inspired by the human brain.
Includes feedforward, backpropagation networks, etc.
Used in classification, regression, and prediction problems.

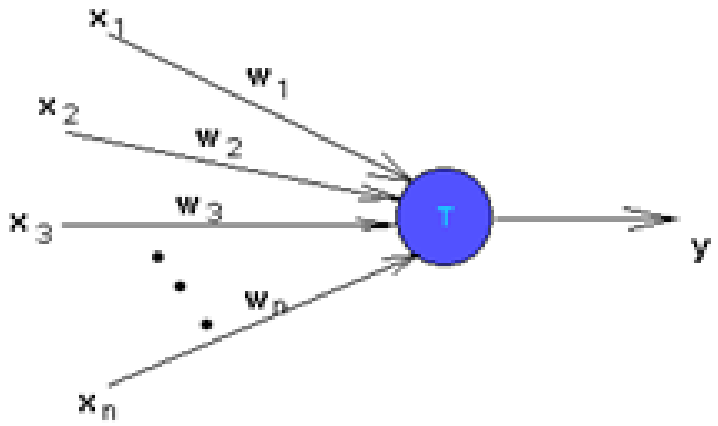
6. Deep Neural Network (DNN)
Neural networks with multiple hidden layers.
Capable of learning complex patterns.
Used in speech recognition, AI systems, and robotics.

Artificial neurons





y = w1*x1 + w2*x2
Take X=1, ==> y=w1*x1+w2 ??



Artificial neurons

Nonlinear generalization of neuron:

$$y = f(x, w)$$

y is the neuron's output, x is the vector of inputs,

and w is the vector of synaptic weights.

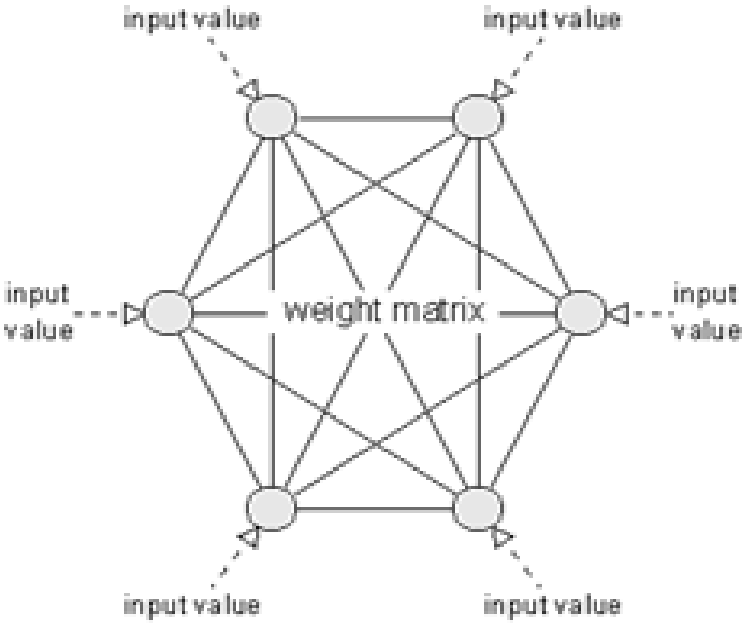
Examples:

$$y = \frac{1}{1 + e^{-w^T x - a}}$$ sigmoidal neuron

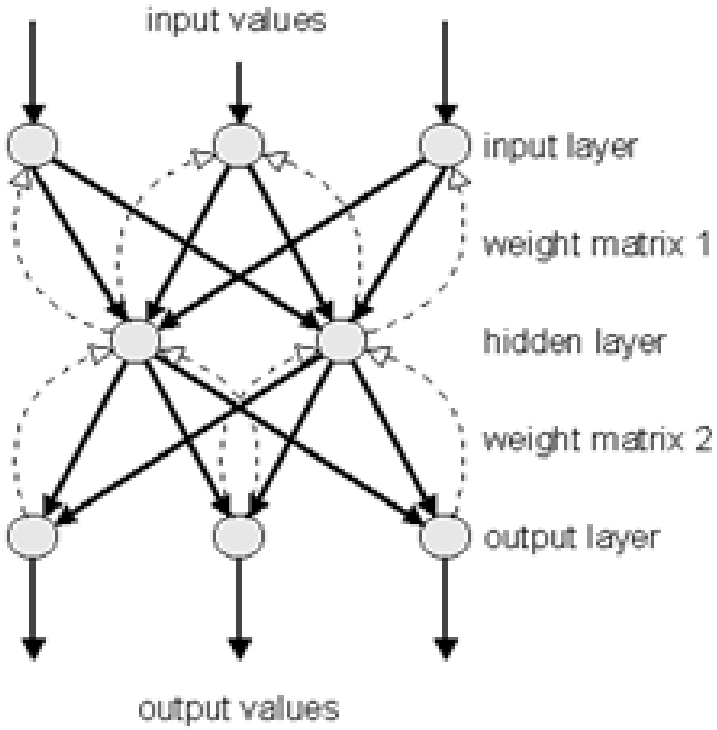
$$y = e^{-\frac{\|x - w\|^2}{2a^2}}$$ Gaussian neuron

Other Model

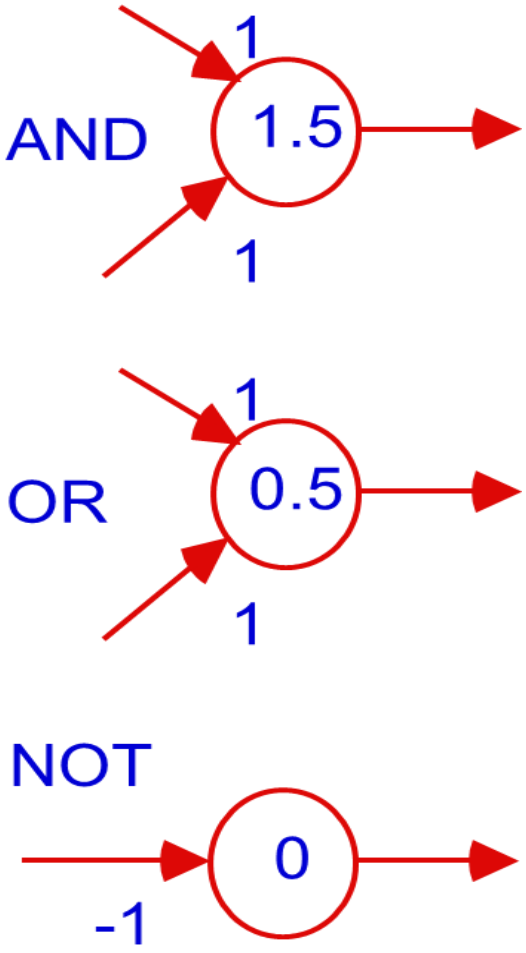
Hopfield



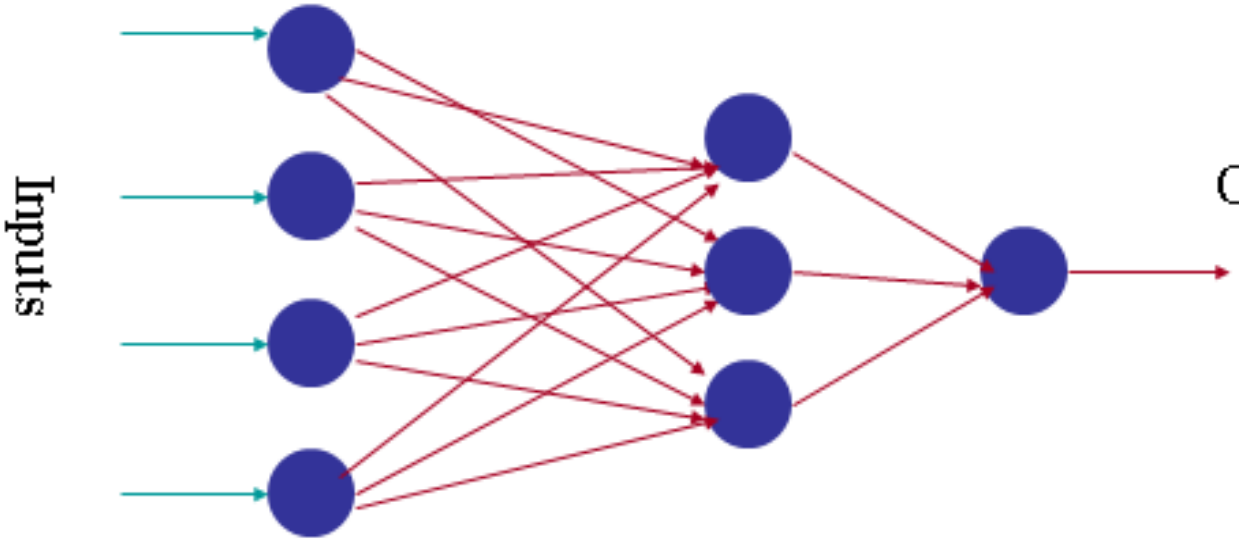
Retropropagation



From Logical Neurons to Finite Automata



Neural network mathematics



$$y_1^1 = f(x_1, w_1^1)$$
$$y_2^1 = f(x_2, w_2^1)$$
$$y_3^1 = f(x_3, w_3^1)$$
$$y_4^1 = f(x_4, w_4^1)$$

$$y^1 = \begin{pmatrix} y_1^1 \\ y_2^1 \\ y_3^1 \\ y_4^1 \end{pmatrix}$$

$$y_1^2 = f(y^1, w_1^2)$$
$$y_2^2 = f(y^1, w_2^2)$$
$$y_3^2 = f(y^1, w_3^2)$$

$$y^2 = \begin{pmatrix} y_3^2 \\ y_3^2 \\ y_3^2 \end{pmatrix}$$

$$y_{Out} = f(y^2, w_1^3)$$

Neural network mathematics

Neural network: input / output transformation

$$y_{out} = F(x, W)$$

W is the matrix of all weight vectors.

Neuron Model

- Firing Rules:
  - Threshold rules:
    - Calculate weighted average of input
    - Fire if larger than threshold
  - Perceptron rule
    - Calculate weighted average of input
    - Output activation level is

$$\phi(v) = \begin{cases} 1 & v \geq \frac{1}{2} \\ v & 0 \leq v \leq \frac{1}{2} \\ 0 & v \leq 0 \end{cases}$$

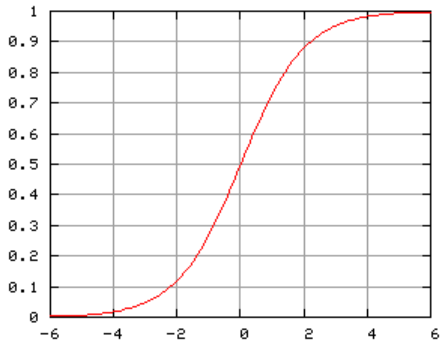
Neuron Model

- Firing Rules: Sigmoid functions:
  - Hyperbolic tangent function

$$\varphi(v) = \tanh(v/2) = \frac{1-\exp(-v)}{1+\exp(-v)}$$

  - Logistic activation function

$$\varphi(v) = \frac{1}{1+\exp(-v)}$$



ANN Forward Propagation

- Apply input vector **X** to layer of neurons.
- Calculate

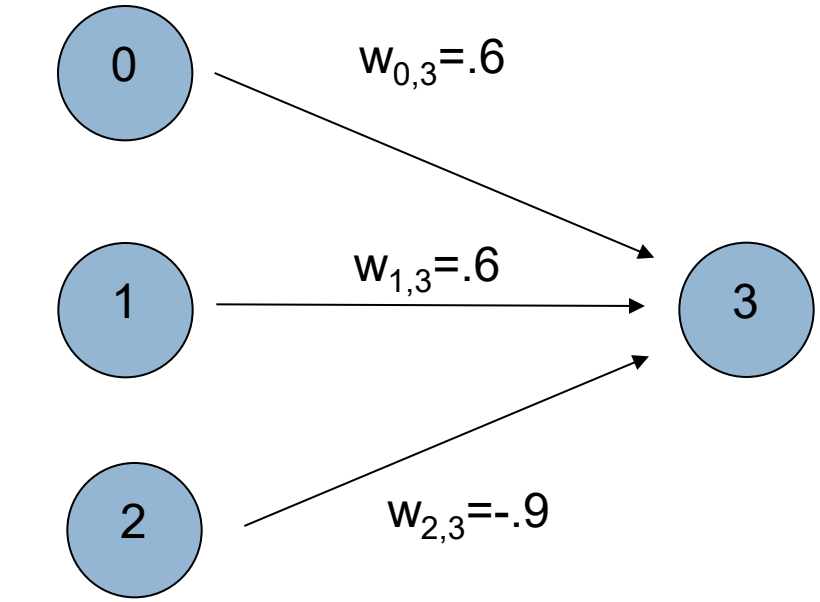
$$V_j(n) = \sum_{i=1}^{p} (W_{ji} X_i + Threshold)$$

  - where X_i is the activation of previous layer neuron i
  - W_{ji} is the weight of going from node i to node j
  - p is the number of neurons in the previous layer
- Calculate output activation

$$Y_j(n) = \frac{1}{1 + \exp(-V_j(n))}$$

ANN Forward Propagation

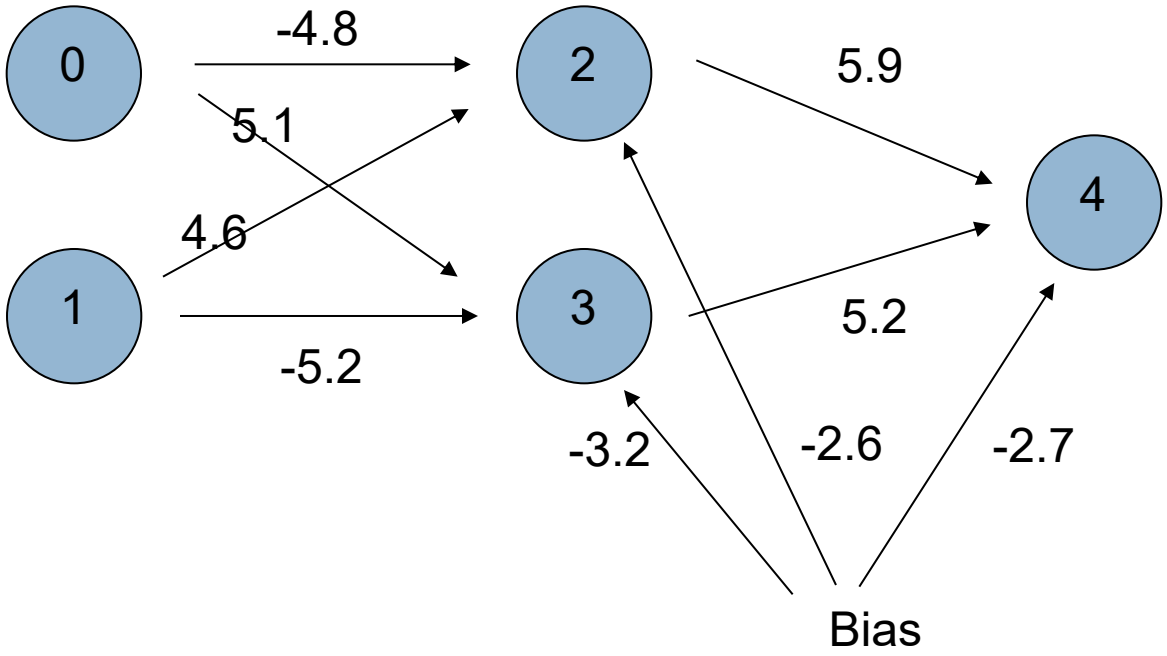
- Example: ADALINE Neural Network
  - Calculates and of inputs



threshold function is step function

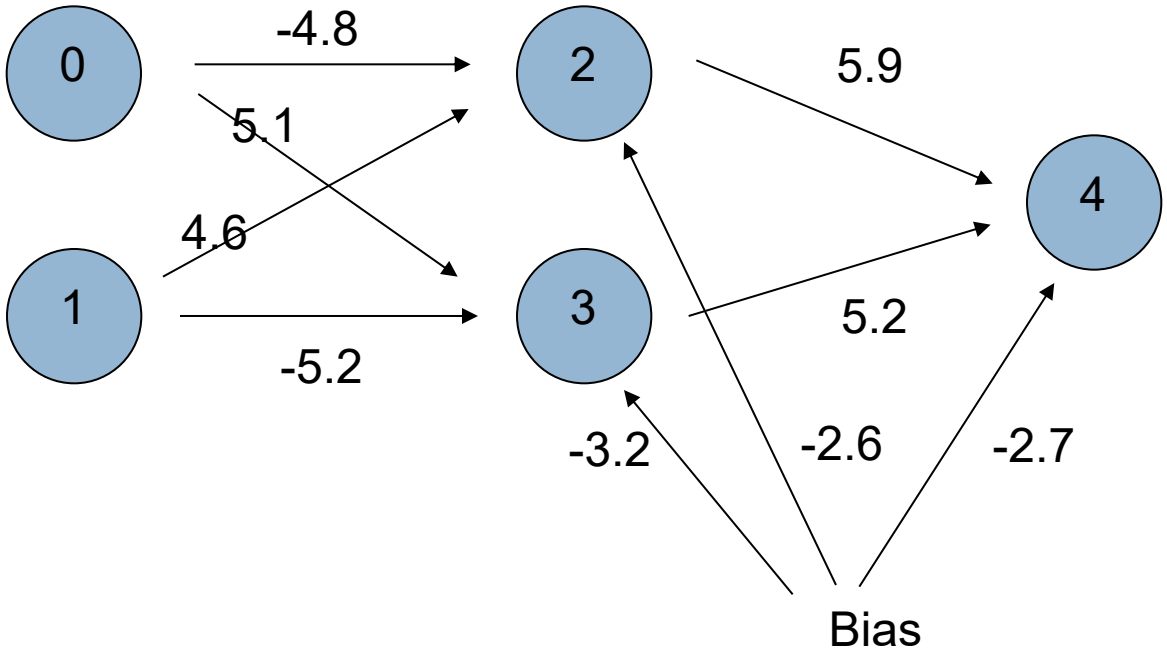
ANN Forward Propagation

- Example: Three layer network
  - Calculates xor of inputs



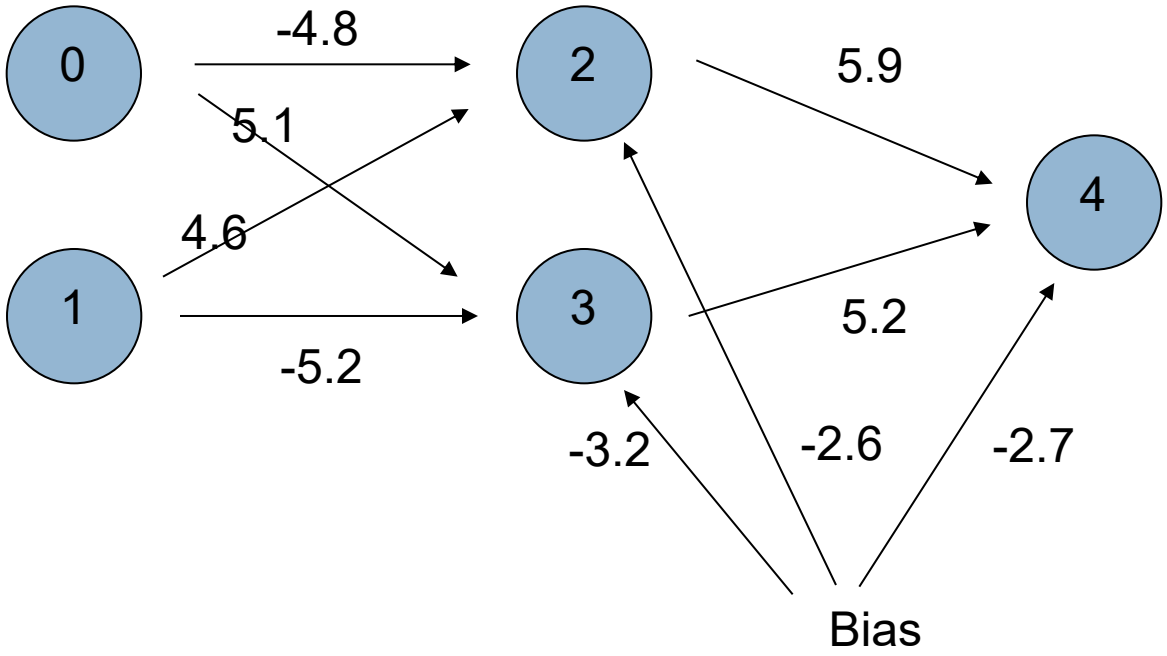
ANN Forward Propagation

- Input (0,0)



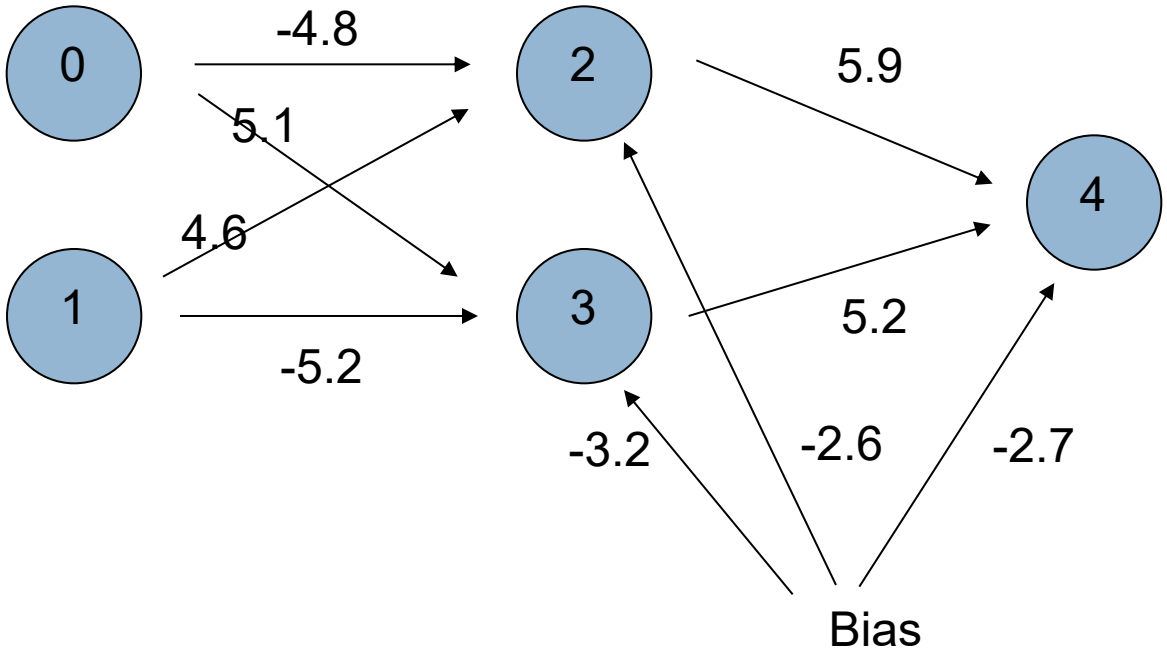
ANN Forward Propagation

- Input (0,0)
  - Node 2 activation is $\varphi(-4.8 \cdot 0 + 4.6 \cdot 0 - 2.6) = 0.0691$



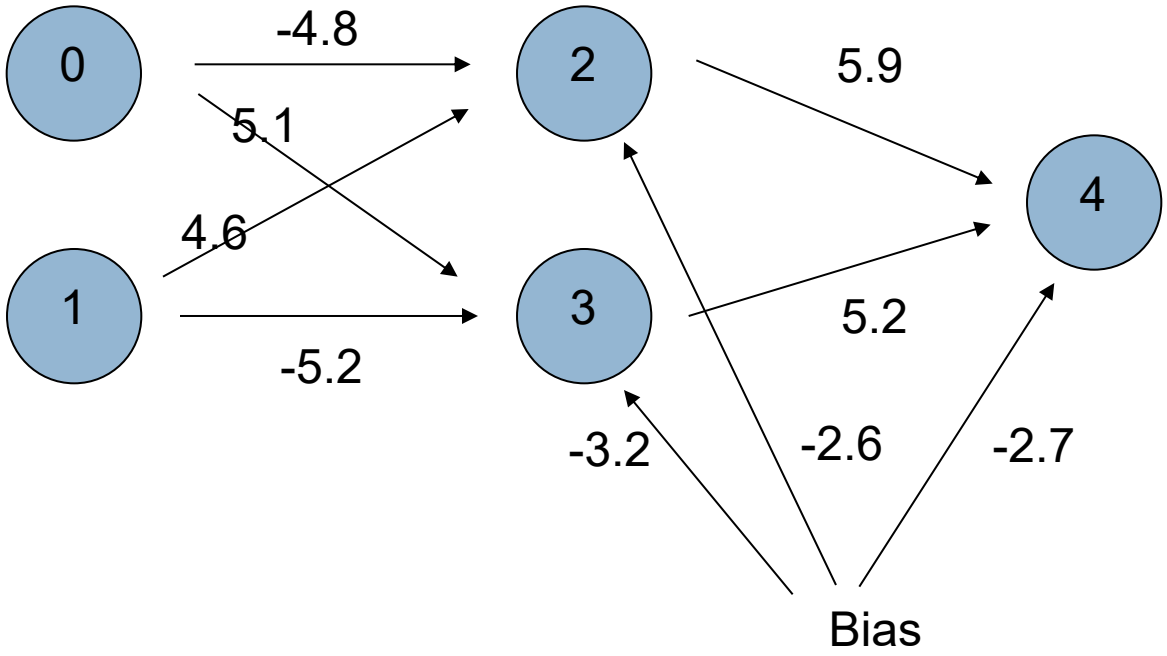
ANN Forward Propagation

- Input (0,0)
  - Node 3 activation is $\varphi(5.1 \cdot 0 - 5.2 \cdot 0 - 3.2) = 0.0392$



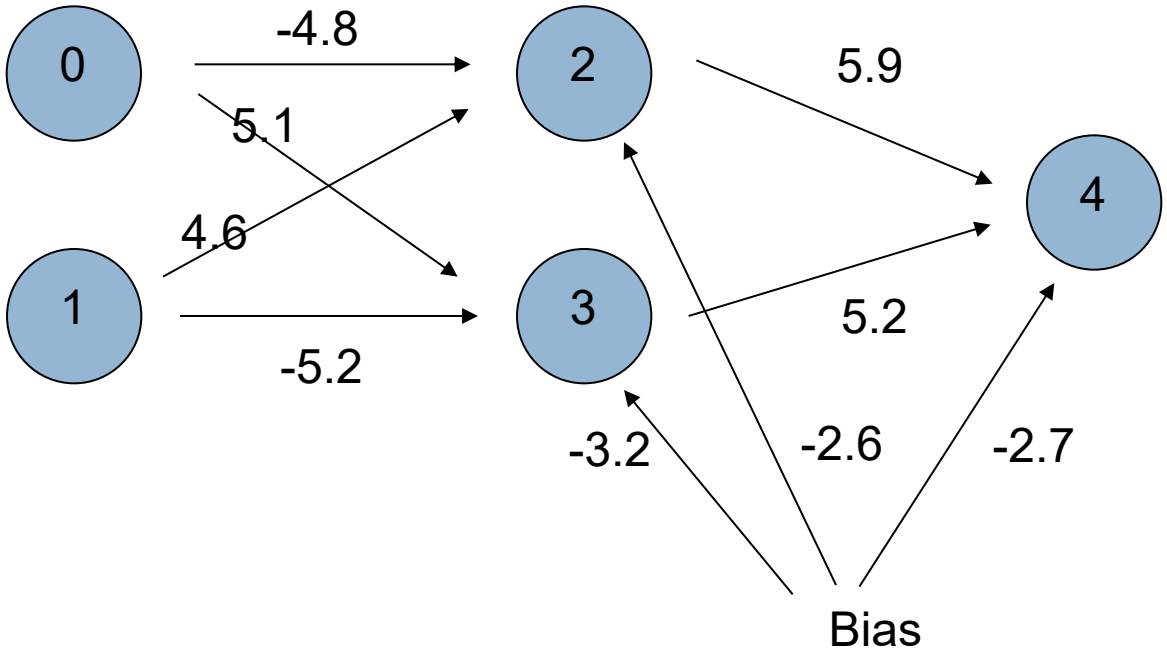
ANN Forward Propagation

- Input (0,0)
  - Node 4 activation is $\varphi(5.9 \cdot 0.069 + 5.2 \cdot 0.069 - 2.7) = 0.110227$



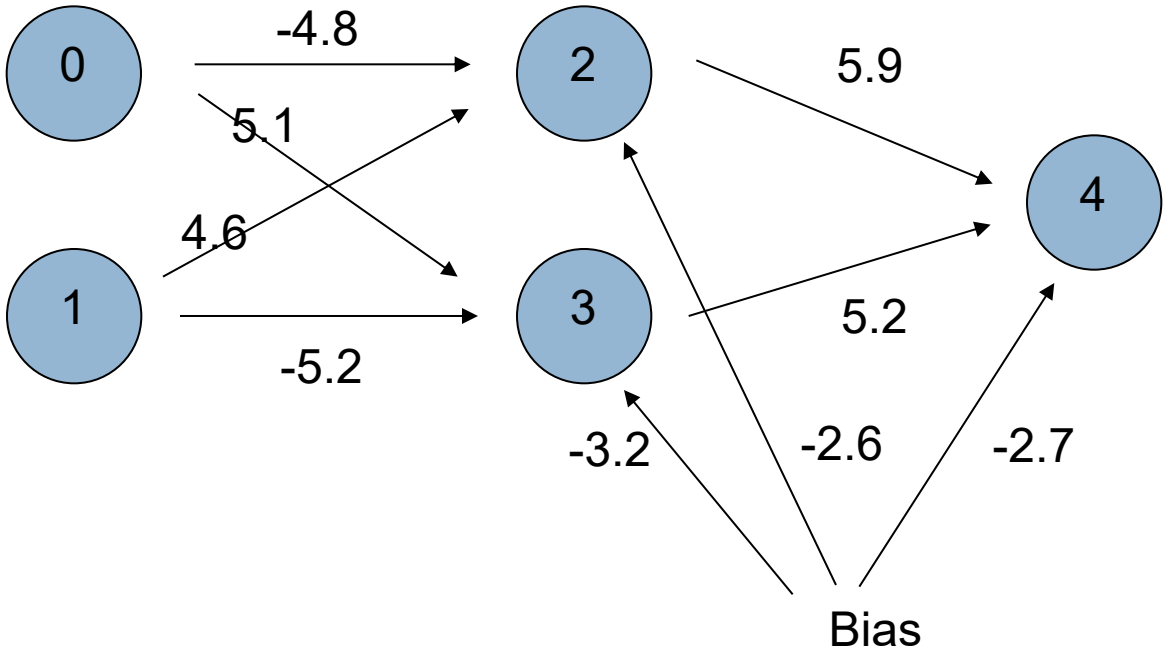
ANN Forward Propagation

- Input (0,1)
  - Node 2 activation is $\varphi(4.6 \cdot -2.6) = 0.153269$



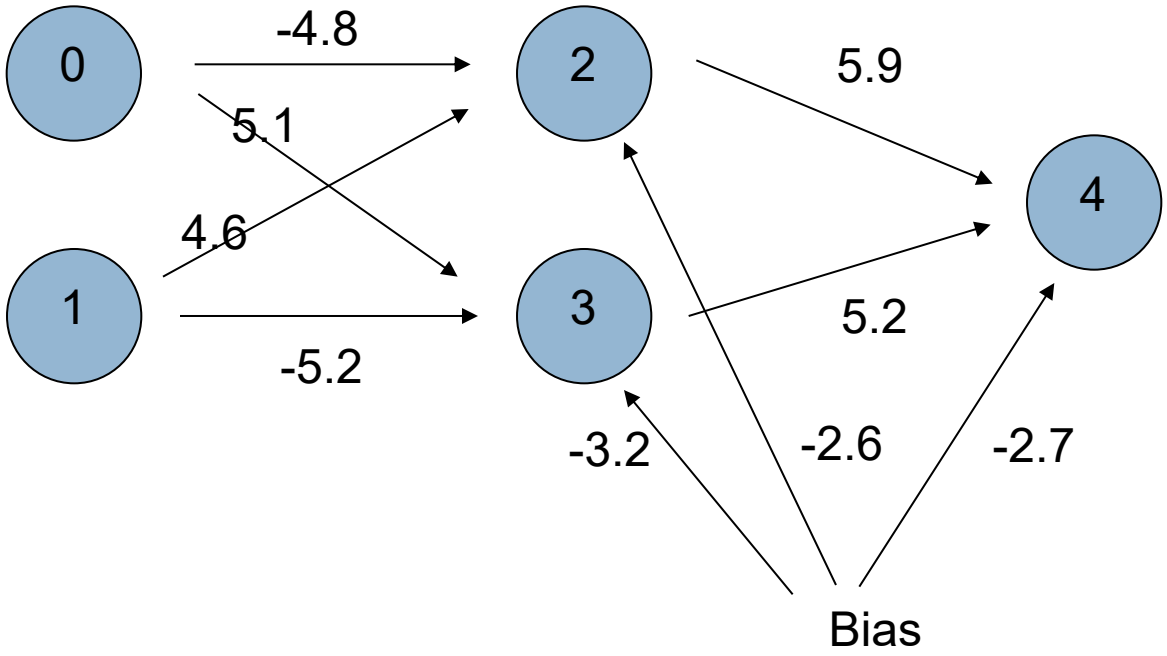
ANN Forward Propagation

- Input (0,1)
  - Node 3 activation is $\varphi(-5.2 \ -3.2) = \ 0.000224817$



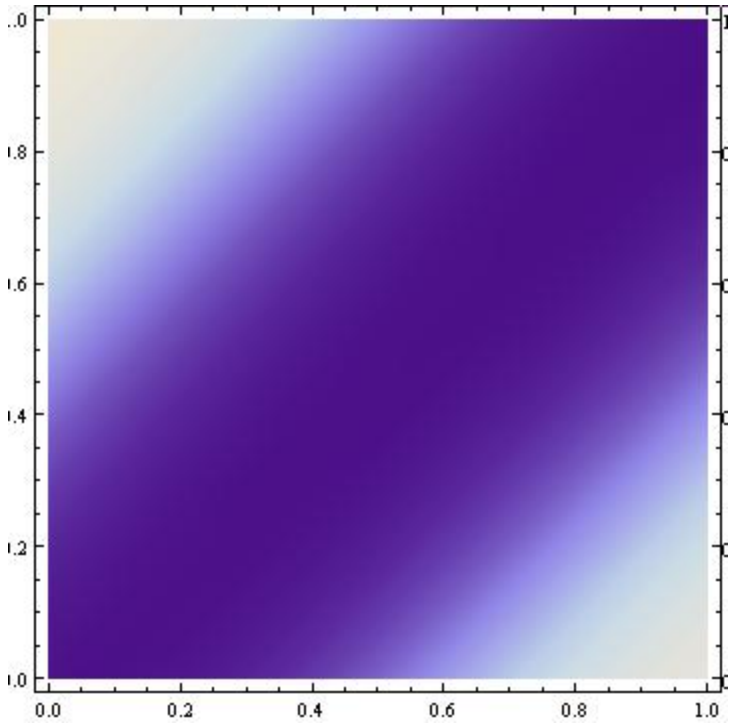
ANN Forward Propagation

- Input (0,1)
  - Node 4 activation is $\varphi(5.9 \cdot 0.153269 + 5.2 \cdot 0.000224817 - 2.7) = 0.923992$

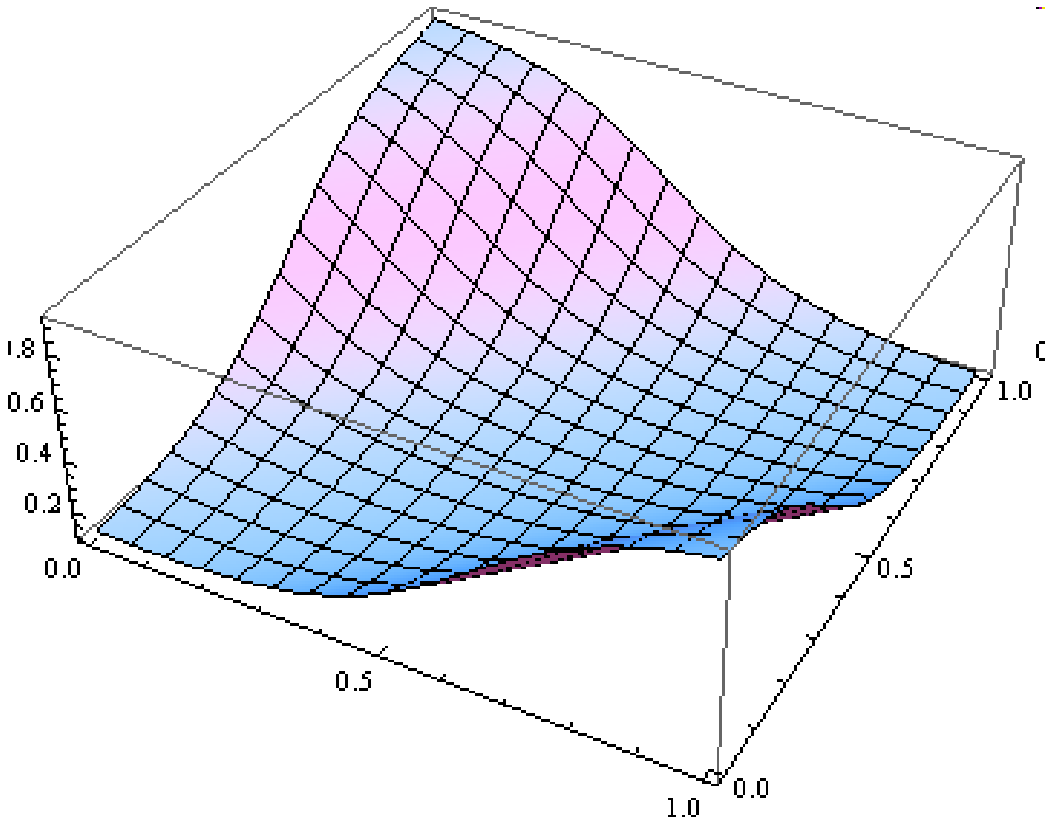


ANN Forward Propagation

- Density Plot of Output



ANN Forward Propagation



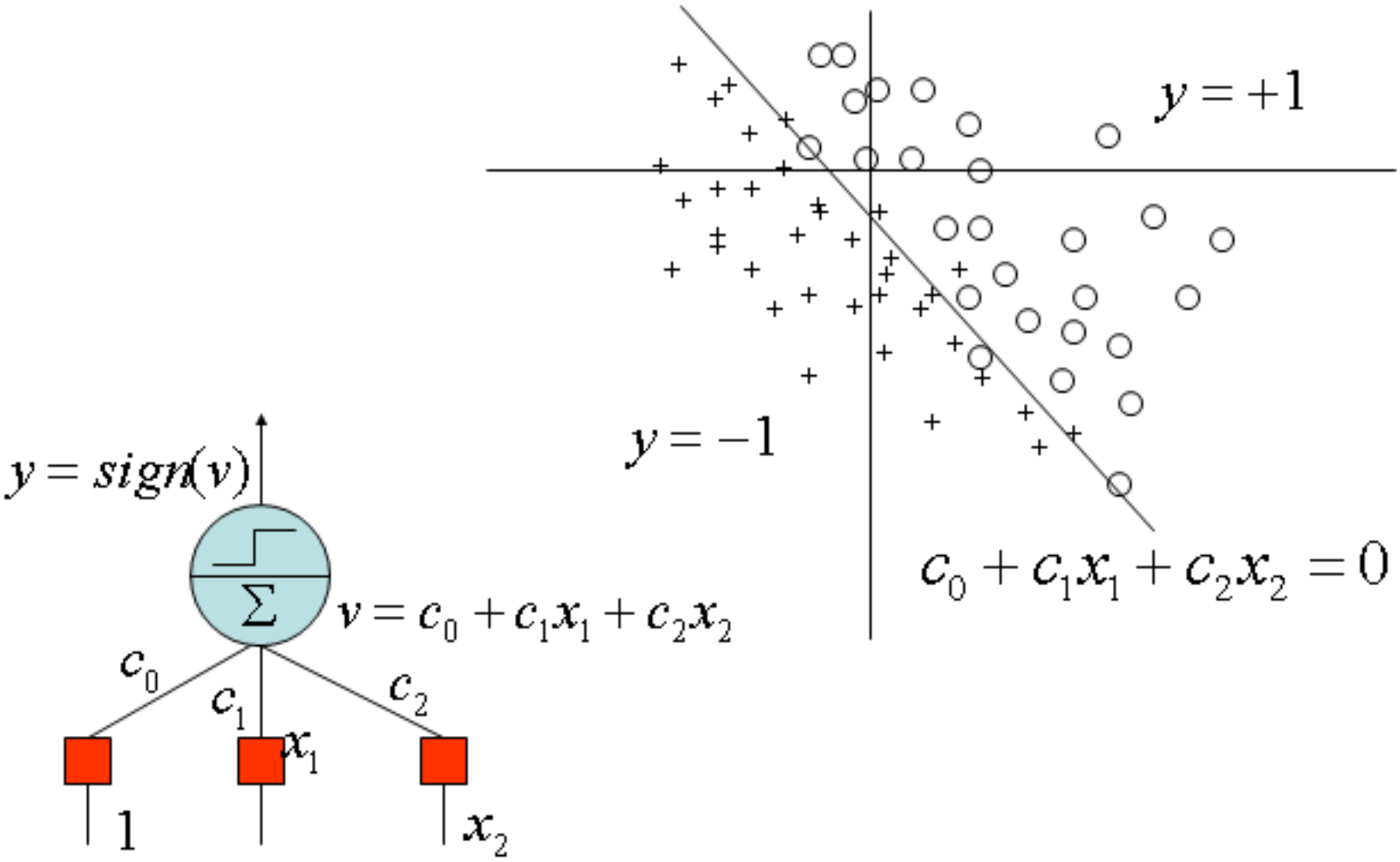
Learning principle for artificial neural networks

- ENERGY MINIMIZATION

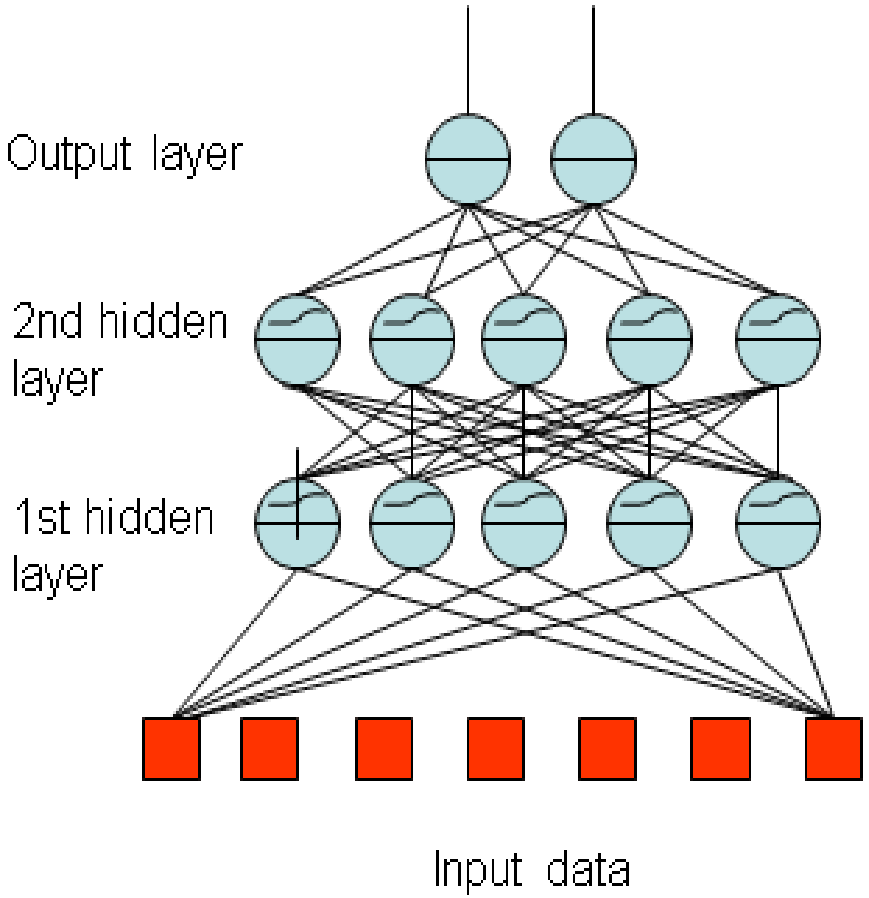
- We need an appropriate definition of energy for artificial neural networks, and having that we can use mathematical optimisation techniques to find how to change the weights of the synaptic connections between neurons.

- ENERGY = measure of task performance error

Perceptron application

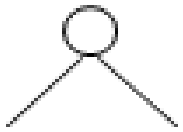
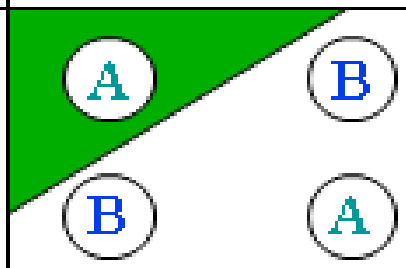
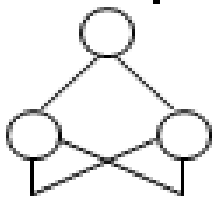
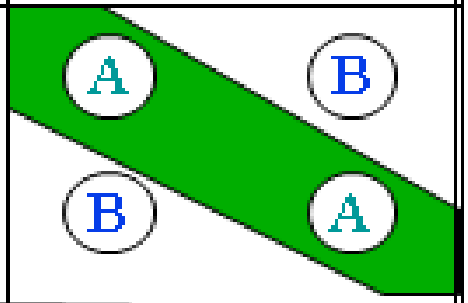
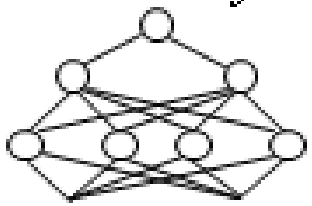
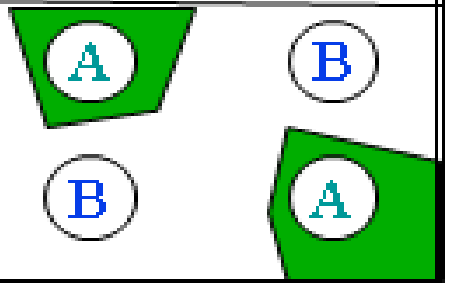


Multi-Layer Perceptron



- One or more hidden layers
- Sigmoid activations functions

Multi-Layer Perceptron Application

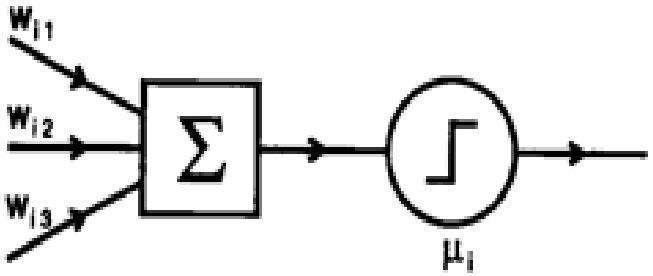
Structure	*Types of Decision Regions*	*Result*
Single-Layer 	*Half Plane Bounded By Hyperplane*	
Two-Layer 	*Convex Open Or Closed Regions*	
Three-Layer 	**Abitrary (Complexity Limited by No. of Nodes)**	

- NN have some **disadvantages** such as:

- **Preprocessing**
- **Results interpretation by high dimension**
- **Learning phase/Supervised/Non Supervised**

McCulloch–Pitts Neural Networks

- Synchronous discrete time operation
  - Time quantized in units of synaptic delay



$n_i \equiv$ output of unit i

$\Theta \equiv$ step function

$w_{ij} =$ weight from unit j to i

$\mu_i =$ threshold

- Output is 1 if and only if weighted sum of inputs is greater than threshold
  $\Theta(x) = 1$ if $x \geq 0$ and 0 if $x < 0$

- Behavior of network can be simulated by a finite automaton
- Any FA can be simulated by a McCulloch-Pitts Network

Properties of Artificial Neural Networks

- High level abstraction of neural input-output transformation
  Inputs → weighted sum of inputs → nonlinear function → output
  - Typically no spikes
  - Typically use implausible constraints or learning rules
- Often used where data or functions are uncertain
  - Goal is to learn from a set of training data
  - And to generalize from learned instances to new unseen data
- Key attributes
  - Parallel computation
  - Distributed representation and storage of data
  - **Learning** (networks adapt themselves to solve a problem)
  - Fault tolerance (insensitive to component failures)

Networks Types

- **Feedforward versus recurrent networks**
  - **Feedforward:** No loops, input → hidden layers → output
  - **Recurrent:** Use feedback (positive or negative)
- **Continuous versus spiking**
  - Continuous networks model mean spike rate (firing rate)
    - Assume spikes are integrated over time
  - Consistent with rate-code model of neural coding
- **Supervised versus unsupervised learning**
  - Supervised networks use a "teacher"
    - The desired output for each input is provided by user
  - Unsupervised networks find hidden statistical patterns in input data
    - Clustering, principal component analysis

Activation functions

Explain different type of activation function. (CO1)

Activation functions are mathematical functions used in neural networks to introduce **non-linearity**, decide neuron output, and help the network learn complex patterns. Below are the **main types of activation functions**, explained simply (exam-oriented):

1. Binary Step Function

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Use: Simple threshold-based decision.
Limitation: Not differentiable → not suitable for backpropagation.

2. Linear (Identity) Function

f(x) = x
Use: Output layer for regression problems.
Limitation: No non-linearity → network behaves like a single-layer model.

Activation functions

3. Sigmoid (Logistic) Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

Range: (0, 1)
Use: Binary classification, probability output.
Advantages: Smooth and differentiable.
Disadvantages: Vanishing gradient problem, slow learning.

4. Tanh (Hyperbolic Tangent) Function

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Range: (−1, 1)
Advantages: Zero-centered output → better than sigmoid.
Disadvantages: Still suffers from vanishing gradient.

Activation functions

5. ReLU (Rectified Linear Unit)

f(x)=max(0,x)
Use: Most commonly used in hidden layers.
Advantages: Fast computation, reduces vanishing gradient.
Disadvantages: Dying ReLU problem (neurons may stop activating).

6. Leaky ReLU

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

Advantage: Solves dying ReLU problem.
α: Small constant (e.g., 0.01).

Activation functions

7. Softmax Function

$$f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Use: Multi-class classification output layer.
Property: Outputs sum to 1 (probability distribution).

Activation functions

Activation Function	Range	Common Use
Binary Step	0 or 1	Simple classifiers
Linear	$(-\infty, \infty)$	Regression output
Sigmoid	(0, 1)	Binary classification
Tanh	(−1, 1)	Hidden layers
ReLU	$[0, \infty)$	Hidden layers
Leaky ReLU	$(-\infty, \infty)$	Improved ReLU
Softmax	(0, 1)	Multi-class output

What are the different factors affecting Back propagation ? Also list the application of Back Propagation Network.

Factors Affecting Back Propagation

Back Propagation learning performance in neural networks depends on several factors:

Learning Rate (η)

Controls the size of weight updates.
Too high → unstable learning.
Too low → slow convergence.

Initial Weights

Poor initialization may lead to slow learning or local minima.
Random small values are preferred.

Activation Function

Sigmoid and tanh may cause vanishing gradient.
ReLU improves convergence speed.

Factors Affecting Back Propagation

Network Architecture
Number of hidden layers and neurons affects learning capacity and accuracy.

Training Data Quality
Noisy or insufficient data reduces learning efficiency.

Momentum Term
Helps accelerate convergence and avoid local minima.

Number of Epochs
Too few → underfitting.
Too many → overfitting.

Error Surface / Local Minima
Complex error surfaces may trap learning in local minima.

Applications of Back Propagation Network

Back Propagation Networks are widely used in various fields:

- **Pattern Recognition**
  - Handwritten character and face recognition.
- **Speech Recognition**
  - Voice-controlled systems.
- **Image Processing**
  - Image classification and object detection.
- **Medical Diagnosis**
  - Disease prediction and analysis of medical data.
- **Forecasting**
  - Stock market and weather prediction.
- **Control Systems**
  - Robotics and industrial process control.
- **Data Mining**
  - Classification and clustering of large datasets.

Explain all steps in Back Propagation Learning.

Steps in Back Propagation Learning Algorithm

Back Propagation is a **supervised learning algorithm** used to train multilayer neural networks. The learning process consists of the following steps:

Step 1: Initialization

Initialize all **weights and biases** to small random values.
Choose **learning rate (η)** and stopping criteria (error limit or number of epochs).

Step 2: Input Pattern Presentation

Apply the **input vector** to the input layer.
Inputs are passed to the hidden layer neurons.

Steps in Back Propagation Learning Algorithm

Step 3: Forward Propagation

Compute the **net input** to each neuron:

$$net_j = \sum w_{ij} x_i + b_j$$

Apply activation function to get output:

$$y_j = f(net_j)$$

Repeat this process layer by layer until the **output layer** is reached.

Step 4: Error Computation

Compare actual output with desired output.
Calculate error:

$$E = \frac{1}{2} \sum (t_k - y_k)^2$$

where t_k is target output and y_k is actual output.

Steps in Back Propagation Learning Algorithm

Step 5: Backward Propagation of Error

Compute error gradient at the output layer:

$$\delta_k = (t_k - y_k) f'(net_k)$$

Propagate error backward to hidden layers:

$$\delta_j = f'(net_j) \sum \delta_k w_{jk}$$

Step 6: Weight and Bias Update

Update weights using gradient descent:

$$w_{new} = w_{old} + \eta \delta x$$

Update biases similarly.

Steps in Back Propagation Learning Algorithm

Step 7: Iteration and Convergence

- Repeat steps 2–6 for all training samples.
- Continue for multiple epochs until:
  - Error becomes minimum, or
  - Maximum epochs are reached.

Step 8: Testing

After training, test the network using unseen data to evaluate performance.

Flow Summary

- ❖ Initialize weights
- ❖ Forward pass
- ❖ Compute error
- ❖ Backward pass
- ❖ Update weights
- ❖ Repeat until convergence

Explain different types of Learning in ANN. (CO2)

Types of Learning in Artificial Neural Networks (ANN)

Learning in ANN refers to the method by which a neural network **adjusts its weights** to improve performance. The main types of learning are explained below:

1. Supervised Learning

- The network is trained using **labeled data** (input–output pairs).
- Error is calculated by comparing actual output with desired output.
- Weights are updated to minimize error.

Examples:

Back Propagation Network
Perceptron learning

Applications:

Pattern recognition
Classification and regression

Types of Learning in Artificial Neural Networks (ANN)

2. Unsupervised Learning

- The network is trained using **unlabeled data**.
- Learns patterns and structure automatically.
- No target output is provided.

Examples:
Self-Organizing Maps (SOM)
Competitive learning

Applications:
Clustering
Data compression

Types of Learning in Artificial Neural Networks (ANN)

3. Reinforcement Learning

- Learning is based on **feedback from the environment**.
- Uses reward and penalty instead of exact target values.
- Learner improves by trial and error.

Examples:
Q-learning
Actor–Critic models

Applications:
Robotics
Game playing

Types of Learning in Artificial Neural Networks (ANN)

4. Semi-Supervised Learning

Uses **both labeled and unlabeled data**.
Useful when labeled data is limited.

Applications:

Speech recognition
Image classification

5. Online Learning

Weights are updated **after each input sample**.
Suitable for real-time systems.

6. Batch Learning

Weights are updated **after processing the entire dataset**.
Stable but slower compared to online learning.

Explain Mc-Cullach Pitts Model with realization of AND & OR Gate. (CO2)

McCulloch–Pitts (M–P) Neuron Model

The **McCulloch–Pitts model** is the **earliest artificial neuron model**, proposed to explain how a biological neuron works using a **binary threshold logic unit**.

Basic Concept

Inputs are **binary** (0 or 1)
Weights are **fixed**
Uses a **threshold activation function**
Output is **binary** (0 or 1)

Mathematical Model

For inputs $x_1, x_2, \ldots, x_n$ with weights $w_1, w_2, \ldots, w_n$:

$$y = \begin{cases} 1, & \sum w_i x_i \geq \theta \\ 0, & \sum w_i x_i < \theta \end{cases}$$

where

- θ = threshold
- y = output

McCulloch–Pitts (M–P) Neuron Model

Assumptions of M–P Model

1. Inputs and outputs are binary.
2. Weights are constant.
3. No learning mechanism.
4. Single neuron decision-making.

McCulloch–Pitts (M–P) Neuron Model

Realization of AND Gate using M–P Neuron:

Truth Table (AND Gate)

x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Parameters

- $w_1 = 1, w_2 = 1$
- Threshold $\theta = 2$

Operation

$$\text{Net input} = x_1 + x_2$$

- Output = 1 only when both inputs are 1

McCulloch–Pitts (M–P) Neuron Model

Realization of OR Gate using M–P Neuron:

Truth Table (OR Gate)

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

Parameters

- $w_1 = 1, w_2 = 1$
- Threshold $\theta = 1$

Operation

$$\text{Net input} = x_1 + x_2$$

- Output = 1 when **any one input is 1**

McCulloch–Pitts (M–P) Neuron Model

Advantages

Simple and easy to understand

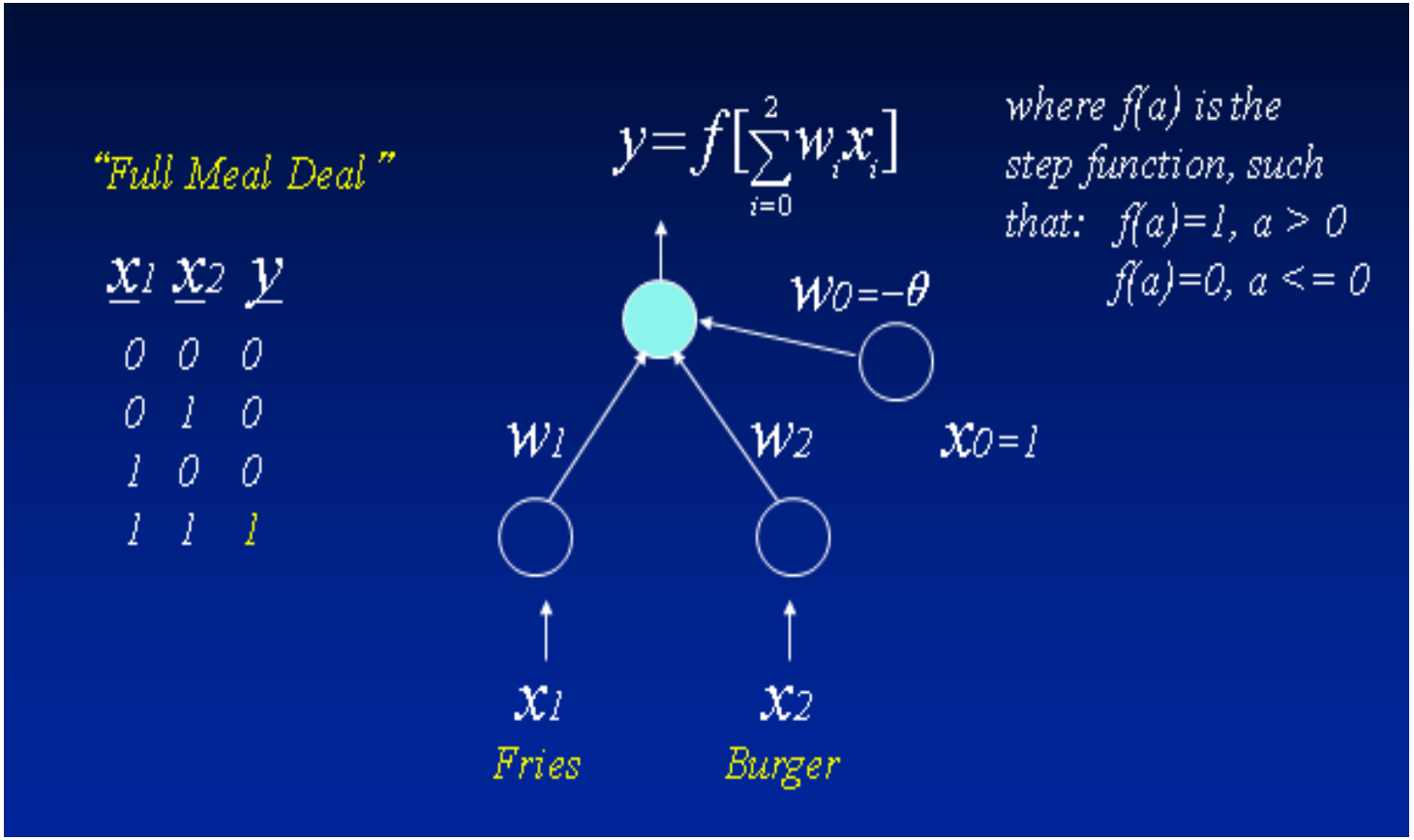
Basis for modern neural networks

Limitations

Cannot learn (no weight update)

Cannot solve non-linearly separable problems (e.g., XOR)

Learning in a Simple Neuron



$$H = \{W \mid W \in R^{(n+1)}\}$$

Perceptron Learning Algorithm:

1. Initialize weights

2. Present a pattern and target output

$$y = f\left[\sum_{i=0}^{2} w_i x_i\right]$$

3. Compute output :

$$w_i(t+1) = w_i(t) + \Delta w_i$$

4. Update weights :

Repeat starting at 2 until acceptable level of error

Widrow-Hoff or Delta Rule for Weight Modification

$$w_i(t+1) = w_i(t) + \Delta w_i \quad ; \quad \Delta w_i = \eta d x_i(t)$$

Where:

η = *learning rate* (o < η <= *1*), typically set = *0.1*

d = *error signal* = desired output - network output

= *t - y*

- Perceptron Learning - A Walk Through

- The PERCEPT.XLS table represents 4 iterations through training data for “full meal deal” network
- On-line weight updates
- Varying learning rate, h , will vary training time

Limitations of Simple Neural Networks

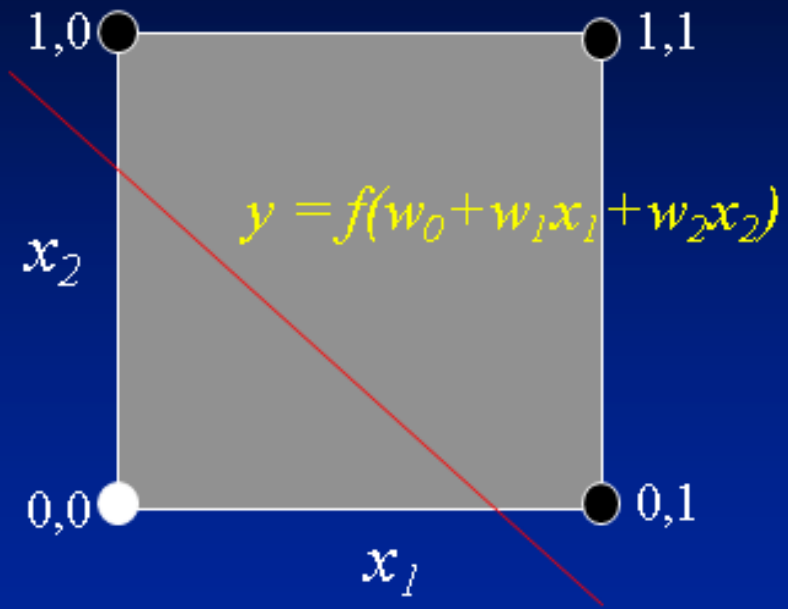
- What is a Perceptron doing when it learns?
- We will see it is often good to visualize network activity
- A discriminate function is generated
- Has the power to map input patterns to output class values
- For 3-dimensional input, must visualize 3-D space and 2-D hyper-planes

EXAMPLE

Simple
Neural Network

Logical OR
Function

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1



What is an artificial neuron doing when it learns?

- The Limitations of Perceptrons

(Minsky and Papert, 1969)

- Able to form only linear discriminate functions; i.e. classes which can be divided by a line or hyper-plane
- Most functions are more complex; i.e. they are non-linear or not linearly separable
- This crippled research in neural net theory for 15 years ....

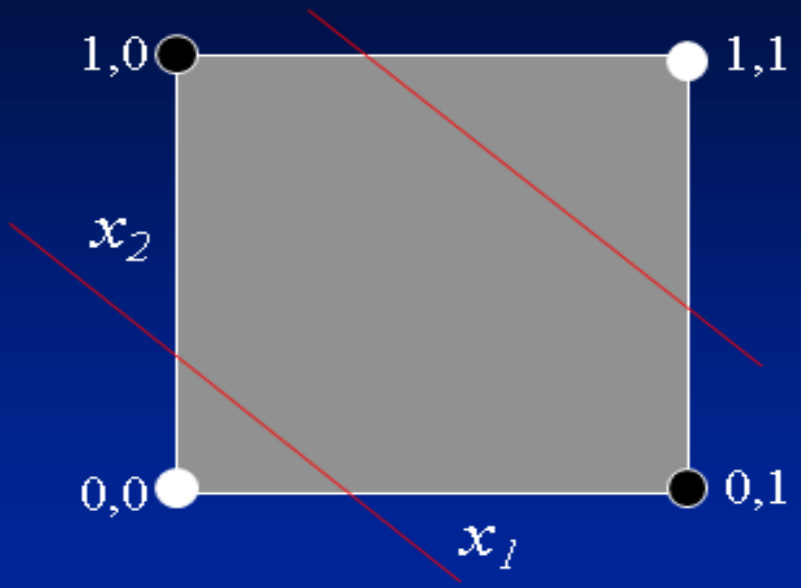
EXAMPLE

Multi-layer Neural Network

Hidden layer of neurons

Logical XOR Function

x1	*x2*	*y*
0	0	0
0	1	1
1	0	1
1	1	0



Two neurons are need! Their combined results can produce good classification.

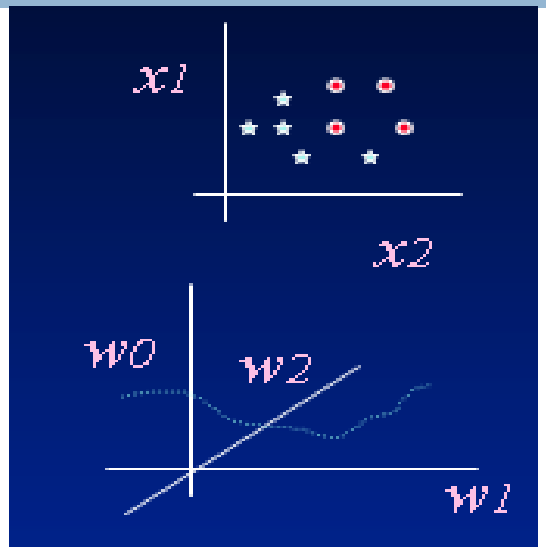
Multi-layer Feed-forward ANNs

- Over the 15 years (1969-1984) some research continued ...
- hidden layer of nodes allowed combinations of linear functions
- non-linear activation functions displayed properties closer to real neurons:
- output varies continuously but not linearly
- differentiable .... Sigmoid $f(a)=1/(1+e^{-a})$
- ➡ non-linear ANN classifier was possible

- However ... there was no learning algorithm to adjust the weights of a multi-layer network - weights had to be set by hand.

- How could the weights below the hidden layer be updated?

- Pattern Space
- Weight Space



- Visualizing the process of learning
  - function surface in weight space
  - error surface in weight space

The Back-propagation Algorithm

- 1986: the solution to multi-layer ANN weight update rediscovered
- Conceptually simple - the global error is backward propagated to network nodes, weights are modified proportional to their contribution
- Most important ANN learning algorithm
- Become known as back-propagation because the error is send back through the network to correct all weights

- Like the Perceptron - calculation of error is based on difference between target and actual output:

$$E = \frac{1}{2}\sum_j (t_j - o_j)^2$$

- However in BP it is the rate of change of the error which is the important feedback through the network

generalized delta rule $\Delta w_{ij} = -\eta \frac{\delta E}{\delta w_{ij}}$

- Relies on the sigmoid activation function for communication

Objective: compute $\frac{\delta E}{\delta w_{ij}}$ for all w_{ij}

Definitions:

w_{ij} = weight from node i to node j

x_j = totaled weighted input of node $= \sum_{i=0}^{n} w_{ij} o_i$

o_j = output of node $= f(x_j) = 1/(1+e^{-x_j})$

E = error for 1 pattern over all output nodes

Objective: compute $\frac{\delta E}{\delta w_{ij}}$ for all w_{ij}

Four step process:

1. Compute how fast error changes as output of node j is changed
2. Compute how fast error changes as total input to node j is changed
3. Compute how fast error changes as weight w_{ij} coming into node j is changed
4. Compute how fast error changes as output of node i in previous layer is changed

On-Line algorithm:

1. Initialize weights
2. Present a pattern and target output
3. Compute output : $o_j = f[\sum_{i=0}^{n} w_{ij} o_i]$
4. Update weights : $w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}$

where $\Delta w_{ij} = -\eta \frac{\delta E}{\delta w_{ij}}$

Repeat starting at 2 until acceptable level of error

Where: $$\Delta w_{ij} = \eta d_j o_i = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta E W_{ij}$$

For output nodes:

$$d_j = EI_j = EA_j o_j (1 - o_j) = o_j (1 - o_j)(t_j - o_j)$$

For hidden nodes:

$$d_i = EI_i = EA_i o_i (1 - o_i) = o_i (1 - o_i) \sum_j EI_j w_{ij}$$

Visualizing the bp learning process:

The bp algorithm performs a *gradient descent* in weights space toward a minimum level of error using a fixed *step size* or learning rate η

The gradient is given by : $\frac{\delta E}{\delta w_{ij}}$

= rate at which error changes as weights change

Momentum Descent:

- Minimization can be speed-up if an additional term is added to the update equation: $\alpha[w_{ij}(t) - w_{ij}(t-1)]$

  where: $o < \alpha < 1$

- Thus: $\Delta w_{ij}(t) = \eta d_j o_i + \alpha \Delta w_{ij}(t-1)$
  - Augments the effective learning rate η to vary the amount a weight is updated
  - Analogous to momentum of a ball - maintains direction
  - Rolls through small local minima
  - Increases weight upadte when on stable gradient

The Back-propagation Algorithm

- Line Search Techniques:
- Steepest and momentum descent use only gradient of error surface
- More advanced techniques explore the weight space using various heuristics
- Most common is to search ahead in the direction defined by the gradient

On-line *vs.* Batch algorithms:

- Batch (or cumulative) method reviews a set of training examples known as an epoch and computes global error:

$$E = \frac{1}{2}\sum_{p}\sum_{j}(t_j - o_j)^2$$

- Weight updates are based on this cumulative error signal
- On-line more stochastic and typically a little more accurate, batch more efficient

The ANN Application Development Process

- Guidelines for using neural networks
- 1. Try the best existing method first
- 2. Get a big training set
- 3. Try a net without hidden units
- 4. Use a sensible coding for input variables
- 5. Consider methods of constraining network
- 6. Use a test set to prevent over-training
- 7. Determine confidence in generalization through cross-validation

Example Applications

- Pattern Recognition (reading zip codes)
- Signal Filtering (reduction of radio noise)
- Data Segmentation (detection of seismic onsets)
- Data Compression (TV image transmission)
- Database Mining (marketing, finance analysis)
- Adaptive Control (vehicle guidance)

Pros and Cons of Back-Prop

- Cons:
- Local minimum - but not generally a concern
- Seems biologically implausible
- Space and time complexity:

  lengthy training times
- It's a black box! I can't see how it's making decisions?
- Best suited for supervised learning
- Works poorly on dense data with few input variables

- Pros:
- Proven training method for multi-layer nets
- Able to learn any arbitrary function (XOR)
- Most useful for non-linear mappings
- Works well with noisy data
- Generalizes well given sufficient examples
- Rapid recognition speed
- Has inspired many new learning algorithms